WaldOT - Digital Twin Engine Documentation

Complete Guide for Users, Developers, and Architects - Version 0.6.2

Written by:	Andrea Ambrosini

andrea.ambrosini@rossonet.com

Zona Autonoma srl

Table of Contents / Indice dei contenuti

1. WaldOT Documentation Hub 3
1.1 Documentation by Audience 3
1.2 Quick Navigation 4
1.3 Key Concepts 4
1.4 Technology Stack 5
1.5 Community & Support 6
1.6 License 6
1.7 Project Sponsor 6
2. Architecture 7
2.1 WaldOT Architecture Overview 7
3. Users & Operators 20
3.1 WaldOT User Guide 20
3.2 WaldOT Configuration Reference 26
3.3 Examples 31
4. Developers 47
4.1 WaldOT Developer Guide 47
4.2 WaldOT Integration Guide: Writing a Plugin 53

1. WaldOT Documentation Hub

Digital Twin Engine bridging OPC UA and Apache TinkerPop Graphs

1.1 Documentation by Audience

WaldOT documentation is organized by user role to provide targeted, relevant information:

1.1.1 🎯 For Decision Makers & Architects

Architecture Overview - Strategic view of WaldOT capabilities

- Business value proposition
- Technical architecture and design patterns
- Integration scenarios and use cases
- Scalability and performance characteristics
- Security and compliance considerations

Target audience: CTOs, Solution Architects, Technical Leads, Project Managers

---

1.1.2 👨‍💻 For Developers

Developer Guide - Hands-on development with WaldOT

- Quick start and setup
- Core concepts and APIs
- Plugin development tutorial
- Code examples and patterns
- Testing and debugging
- Best practices

Integration Guide - How to write a WaldOT plugin (verified step-by-step example)

- Plugin architecture deep dive
- Virtual threads and concurrency
- OPC UA type registration
- Real-world plugin examples

Target audience: Software Engineers, DevOps Engineers, Integration Specialists

---

1.1.3 👤 For End Users & Operators

User Guide - Operating manual

- Start the server, OPC UA address space
- Vertices, active edges (`link-to` , `link-from` , `fire`), plugin types
- Bootstrap file, commands, Gremlin Server
- Rules engine, monitoring, operations, troubleshooting

Examples - Step-by-step pages verified on the Docker image

- Plain vertices, active edges, generators and rules, complete bootstrap
- Showcases: Dungeon Arena, Evolution Lab, Gas Grid

Configuration Reference - All server options

Target audience: System Administrators, OT Engineers, Operators

---

1.2 Quick Navigation

1.2.1 Getting Started

- Start the server
- Bootstrap file
- Configuration Reference - every command-line option, with defaults

1.2.2 Core Features

- Generator Plugin - Data simulation
- Rules Engine Plugin - Event-driven automation
- TinkerPop Plugin - Gremlin server access

1.2.3 Examples

- Step-by-step examples - plain vertices, active edges, rules, bootstrap
- Showcases: Dungeon Arena, Evolution Lab, Gas Grid
- Docker Compose examples - server deployment variants

1.2.4 Module Documentation

Each module keeps its technical README next to the code:

Module	Content
waldot-api	Public contracts used by runtimes and plugins
waldot-namespace	OPC UA ⇔ graph bridge, Milo strategies, bootstrap (syntax)
waldot-runner	Picocli runner with all server options
waldot-app	Server application, distribution and Docker examples

---

1.3 Key Concepts

1.3.1 What is WaldOT?

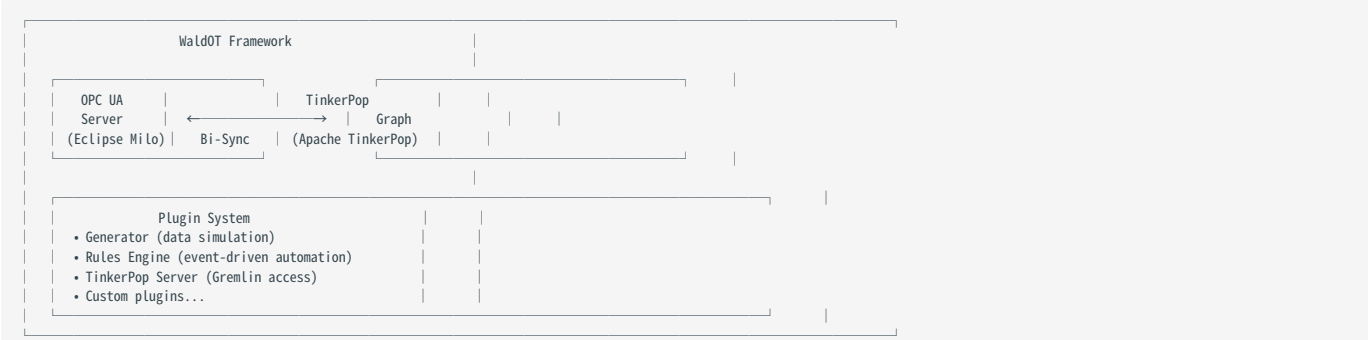
WaldOT is a **Digital Twin Engine** that creates a living bridge between:

- **OPC UA** (industrial automation standard)
- **Apache TinkerPop** (graph database framework)

This enables:

- Real-time bidirectional synchronization between OPC UA and graph databases
- Complex graph queries on industrial data using Gremlin
- Event-driven automation with the rules engine
- Edge computing capabilities for low-latency processing

1.3.2 Core Architecture



1.3.3 Key Features

1. **Unified OT/IT Data Model**
2. OPC UA Objects → TinkerPop Vertices
3. OPC UA References → Graph Edges
4. OPC UA Variables → Vertex Properties
5. **Powerful Graph Queries**
6. Gremlin query language for complex traversals
7. Real-time queries on live industrial data
8. Graph algorithms (PageRank, shortest path, community detection)
9. **Event-Driven Rules Engine**
10. IF-THEN-THAT rules with JEXL expressions
11. Priority-based event processing
12. Hysteresis deduplication
13. Full graph access in rule conditions and actions
14. **Data Generation & Simulation**
15. 6 algorithms (random, sinusoidal, triangular, etc.)
16. Perfect for testing without physical hardware
17. Thousands of concurrent generators with virtual threads
18. **Edge-First Architecture**
19. Runs on-device (RTUs, industrial PCs, edge gateways)
20. Low latency, offline capable
21. Bandwidth efficient

1.4 Technology Stack

- **Java 26** (build target) - virtual threads for massive concurrency

- **Eclipse Milo** - OPC UA server implementation
- **Apache TinkerPop 3.x** - Graph database framework
- **Apache Commons JEXL** - Expression language for rules
- **Docker** - Containerized deployment

1.5 Community & Support

- **GitHub**: rossonet/waldot
- **Docker Hub**: rossonet/waldot
- **Maven Central**: net.rossonet.waldot
- **Issues**: GitHub Issues

1.6 License

WaldOT is released under the **Apache License 2.0**.

1.7 Project Sponsor

Zona Autonoma srl - Industrial IoT and Edge Computing Solutions

2. Architecture

2.1 WaldOT Architecture Overview

Strategic Technical Guide for Decision Makers and Architects

2.1.1 Executive Summary

WaldOT is a Digital Twin Engine that bridges the gap between Operational Technology (OT) and Information Technology (IT) by creating a bidirectional, real-time synchronization between OPC UA industrial systems and Apache TinkerPop graph databases.

Business Value Proposition

- **Unified Data Model**: Single source of truth for industrial and analytical data
- **Real-Time Insights**: Complex graph queries on live OPC UA data without ETL pipelines
- **Edge Computing**: Low-latency processing directly on industrial devices
- **Reduced Integration Costs**: Standard protocols (OPC UA + Gremlin) eliminate custom integrations
- **Scalability**: From single devices to distributed enterprise deployments
- **Future-Proof**: Built on open standards (OPC UA, Apache TinkerPop, Java 26)

2.1.2 Architecture Layers

Layer 1: OPC UA Server (Industrial Interface)

Technology: Eclipse Milo OPC UA SDK

Purpose: Expose industrial data using the OPC UA standard protocol

Capabilities:

- Full OPC UA server implementation (binary protocol, security, subscriptions)
- Standard OPC UA address space with Objects, Variables, References
- Event system for real-time notifications
- Security: X.509 certificates, user authentication, encryption
- Compatible with all OPC UA clients (UaExpert, Prosys, custom applications)

Deployment Scenarios:

- Standalone OPC UA server for legacy system integration
- Gateway between proprietary protocols and OPC UA
- Edge device for local data aggregation

Layer 2: TinkerPop Graph Database (Analytical Engine)

Technology: Apache TinkerPop 3.x

Purpose: Represent industrial data as a queryable graph structure

Capabilities:

- Vertices (nodes) represent industrial objects (machines, sensors, zones)
- Edges (relationships) represent connections (contains, monitors, controls)
- Properties store dynamic data (temperature, pressure, status)
- Gremlin query language for complex traversals
- Graph algorithms (PageRank, shortest path, community detection)
- Multiple storage backends (in-memory, RocksDB, Neo4j, JanusGraph)

Use Cases:

- Correlation analysis across distant equipment
- Root cause analysis with graph traversal
- Asset relationship mapping
- Predictive maintenance pattern detection

Layer 3: Bidirectional Synchronization (Core Innovation)

Purpose: Keep OPC UA and TinkerPop in perfect sync

Synchronization Flow:

```
OPC UA Client → Write Variable → OPC UA Server
                                      ↓
                                  WaldOT Sync
                                      ↓
                                TinkerPop Graph
                                      ↓
                                Gremlin Listeners
                                      ↓
                                Rules Engine / Analytics

Gremlin Query → Modify Graph → WaldOT Sync
                                      ↓
                                OPC UA Server
                                      ↓
                                OPC UA Clients (notified)
```

Key Features:

- **Real-time**: Changes propagate in milliseconds
- **Bidirectional**: Modifications from either side sync to the other
- **Consistent**: ACID properties maintained where supported by storage backend
- **Event-driven**: Observers and listeners for reactive programming

---

Layer 4: Plugin System (Extensibility)

Technology: Java 26 with @WaldotPlugin annotation-based discovery

Purpose: Extend WaldOT with domain-specific functionality

Architecture:

```
┌──────────────────────────────────────────────────────────────────────────┐
│                Plugin Manager                                            │
│   • Auto-discovery (@WaldotPlugin)                                       │
│   • Lifecycle management (init → start → stop)                           │
│   • Type registration in OPC UA namespace                                │
└──────────────────────────────────────────────────────────────────────────┘
                        ↓
┌──────────────────────────────────────────────────────────────────────────┐
│                Available Plugins                                         │
│   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐                 │
│   │  Generator   │   │ Rules Engine │   │ TinkerPop │ │
```

Plugin Capabilities:

- Register custom vertex types (e.g., "PLC", "Sensor", "Actuator")
- Implement custom behaviors (polling, event processing, calculations)
- Integrate external systems (MQTT, Modbus, REST APIs, databases)
- Provide reusable commands accessible via OPC UA methods

Development Effort:

- Simple plugin: 1-2 days
- Complex integration: 1-2 weeks
- Full documentation and examples provided

2.1.3 Core Plugins

1. Generator Plugin

Purpose: Simulate dynamic data for testing and development

Features:

- 6 algorithms: incremental, decremental, random, sinusoidal, triangular, stopped
- Configurable update intervals (10ms to hours)
- Virtual threads for thousands of concurrent generators
- Perfect for testing without physical hardware

Use Cases:

- Development and testing
- Training and demonstrations
- Load testing
- Rule engine validation

Performance: 100,000+ generators on standard hardware

---

2. Rules Engine Plugin

Purpose: Event-driven IF-THEN-THAT automation

Features:

- JEXL expressions for conditions and actions
- Priority-based event queuing
- Hysteresis deduplication (prevent event flooding)
- Full graph access in rules (Gremlin queries)
- Virtual thread execution for concurrency
- Debug events for troubleshooting

Architecture:

```
[Source Node] → [FireMonitoredEdge] → [RuleVertex] → [ComputeMonitoredEdge] → [ComputeVertex]
   (event)         (filters)          (condition)          (routes)            (executes action)
```

Use Cases:

- Real-time alerting (temperature > threshold)
- Complex condition monitoring (multiple sensors)
- Automated responses (shutdown on critical state)
- Data validation and quality checks
- State machine implementation

Performance: Thousands of rules, sub-millisecond latency for simple conditions

---

3. TinkerPop Server Plugin

Purpose: Enable remote Gremlin client access

Features:

- Embedded Gremlin Server as graph vertices
- WebSocket protocol (port 8182 by default)
- GraphSON v3 and GraphBinary v1 serializers
- Compatible with standard TinkerPop clients
- Graph visualization tools support (Graph-Explorer, Gephi)

Use Cases:

- Remote graph queries from applications
- Integration with BI tools
- Graph visualization
- Multi-user access to WaldOT graph

2.1.4 Concurrency Model: Virtual Threads

Technology: Java 26 (Project Loom) Virtual Threads

Why Virtual Threads?

Traditional platform threads:

- ~1 MB stack size per thread
- OS-managed scheduling
- Limited to thousands of threads
- Blocking I/O blocks OS thread

Virtual threads:

- ~1 KB stack size per thread
- JVM-managed scheduling
- Millions of threads possible
- Blocking I/O doesn't block OS thread

Impact on WaldOT:

```
Scenario: 10,000 sensors polling every second

Platform Threads:
- 10,000 threads × 1 MB = ~10 GB RAM
- Context switching overhead
- Not feasible

Virtual Threads:
- 10,000 threads × 1 KB = ~10 MB RAM
- Minimal overhead
- Easily achievable
```

Design Pattern:

```
// Each active object (generator, monitor, rule) runs in its own virtual thread
ExecutorService executor = ThreadHelper.newVirtualThreadExecutor();

// Create thousands of concurrent tasks
for (int i = 0; i < 100000; i++) {
    executor.submit(() -> {
        while (active) {
            // Blocking operations are OK in virtual threads
            data = fetchSensorData();  // Blocking I/O
            updateGraph(data);
            Thread.sleep(1000);  // Blocking sleep
        }
    });
}
```

Benefits:

- Simple, readable code (no async/await complexity)
- Massive concurrency with minimal resources
- Perfect for I/O-bound workloads (sensors, APIs, databases)

2.1.5 Deployment Architectures

Architecture 1: Edge Device

Scenario: Single industrial device (RTU, PLC, edge gateway)

```
┌─────────────────────────────────────────┐
│        Edge Device                      │
│  ┌──────────────────────────────────┐   │
│  │  WaldOT Container                │   │
│  │  • OPC UA Server                 │   │
│  │  • Local graph (RocksDB)         │   │
│  │  • Rules engine                  │   │
│  │  • Data generators               │   │
│  └──────────────────────────────────┘   │
│                                         │
│  Hardware: 1GB RAM, 2 CPU cores         │
└─────────────────────────────────────────┘
        ↓ (optional)
    Cloud/SCADA
```

Characteristics:

- Low latency (<1ms for local rules)
- Offline capable
- Minimal bandwidth usage
- Local data processing

Use Cases:

- Remote sites with limited connectivity
- Real-time control loops
- Edge analytics
- Data aggregation before cloud upload

Architecture 2: Gateway/Aggregator

Scenario: Multiple devices aggregated through WaldOT gateway

Characteristics:

- Protocol translation (Modbus, MQTT, OPC UA)
- Data normalization
- Local buffering
- Bandwidth optimization

Use Cases:

- Legacy system integration
- Multi-protocol environments
- Data preprocessing
- Local analytics before cloud

Architecture 3: Distributed Enterprise

Scenario: Multiple WaldOT instances with centralized graph database

Characteristics:

- Distributed data collection
- Centralized analytics
- Scalable to thousands of devices
- Enterprise-wide visibility

Use Cases:

- Multi-site manufacturing
- Fleet management
- Enterprise asset management
- Centralized monitoring

2.1.6 Integration Scenarios

Scenario 1: OPC UA to Cloud

Problem: Legacy OPC UA devices need cloud connectivity

Solution: WaldOT as OPC UA client + cloud bridge

```
[OPC UA Devices] → [WaldOT OPC UA Client] → [Graph Processing] → [Cloud API]
```

Benefits:

- No changes to existing OPC UA infrastructure
- Data transformation and filtering at edge
- Bandwidth optimization
- Offline buffering

Scenario 2: Graph Analytics on Industrial Data

Problem: Need complex queries on industrial data (e.g., "find all motors connected to Line 1 in alarm state")

Solution: WaldOT exposes OPC UA as queryable graph

```
[OPC UA Data] → [WaldOT Graph] → [Gremlin Queries] → [Analytics / BI Tools]
```

Example Query (Gremlin Server, `waldot-plugin-tinkerpop`):

```
// Find all motors in alarm state connected to Line 1
g.V().has('name', 'Line1')
  .out('contains')
  .has('type', 'motor')
  .has('status', 'alarm')
  .values('name')
```

Benefits:

- No ETL pipeline required
- Real-time queries on live data
- Complex relationship traversal
- Graph algorithms (PageRank, community detection)

Scenario 3: Event-Driven Automation

Problem: Need automated responses to complex conditions

Solution: WaldOT Rules Engine

```
[Sensors] → [WaldOT Graph] → [Rules Engine] → [Automated Actions]
```

Example Rule:

```
// IF temperature > 80 AND pressure > 100 THEN close the inlet valve
var reactor = "g.V().has('id','reactor').next()";
graph.addVertex('id', 'reactor-critical', 'type', 'rules:rule', 'label', 'reactor-critical',
    'condition', reactor + ".value('temperature') > 80.0 && " + reactor + ".value('pressure') > 100.0",
    'action', "g.V().has('id','inlet-valve').next().property('opening', 0.0); log.error('CRITICAL')");
```

The rule runs when a `fire` edge notifies it and a `rules:compute` executes it (see User Guide).

Benefits:

- Low latency (edge processing)
- Complex multi-sensor conditions
- Full graph access in rules
- Priority-based execution

2.1.7 Performance Characteristics

Scalability

Metric	Value	Notes
Vertices	1M+	Depends on storage backend
Edges	10M+	Depends on storage backend
Generators	100K+	Limited by CPU, not memory (virtual threads)
Rules	10K+	Concurrent execution with virtual threads
OPC UA Clients	100+	Standard Eclipse Milo limits
Events/sec	10K+	Depends on rule complexity

Latency

Operation	Latency	Notes
Property update (OPC UA → Graph)	<1ms	In-memory graph
Property update (Graph → OPC UA)	<1ms	In-memory graph
Simple rule execution	<1ms	Boolean condition
Complex rule (Gremlin query)	1-10ms	Depends on query
Graph traversal (local)	<10ms	Depends on complexity

Resource Requirements

Deployment	RAM	CPU	Storage
Minimal (edge)	512MB	1 core	100MB
Standard	2GB	2 cores	1GB
Large (10K vertices)	4GB	4 cores	10GB
Enterprise (100K vertices)	16GB+	8+ cores	100GB+

2.1.8 Security Considerations

OPC UA Security

- **Encryption**: TLS 1.2+ for all OPC UA connections
- **Authentication**: X.509 certificates, username/password
- **Authorization**: Role-based access control (RBAC)
- **PKI**: Full Public Key Infrastructure support
- **Security Policies**: Sign, SignAndEncrypt, None (configurable)

Application Security

- **Default Credentials**: MUST be changed in production (`--factory-password`)
- **Anonymous Access**: Disable in production (`--anonymous-access false`)
- **Exec Command**: Disable in production (`--exec-command-executable false`)
- **Network Isolation**: Bind to specific interfaces, use firewalls
- **Secrets Management**: Use Docker secrets, Kubernetes secrets, or vault

Compliance

- **IEC 62443**: Industrial security standards compatible
- **GDPR**: No personal data collected by default
- **Audit Logging**: All operations logged via SLF4J

2.1.9 Technology Comparison

WaldOT vs. Traditional SCADA

Feature	WaldOT	Traditional SCADA
Data Model	Graph (flexible)	Hierarchical (rigid)
Query Language	Gremlin (powerful)	SQL (limited relationships)
Real-time	Yes	Yes
Edge Computing	Yes	Limited
Extensibility	Plugin architecture	Vendor-specific
Standards	OPC UA, TinkerPop	Proprietary
Cost	Open-source	Expensive licenses

WaldOT vs. Time-Series Databases

Feature	WaldOT	Time-Series DB
Relationships	Native (graph)	Limited (tags)
Complex Queries	Gremlin traversal	Aggregations only
Real-time Updates	Bidirectional	Write-only
OPC UA Integration	Native	Requires gateway
Graph Algorithms	Yes	No
Time-series Optimization	Via plugins	Native

WaldOT vs. Pure Graph Databases

Feature	WaldOT	Neo4j / JanusGraph
OPC UA Integration	Native	Requires custom code
Industrial Protocols	Via plugins	Not supported
Edge Deployment	Optimized	Heavy
Rules Engine	Built-in	External
Data Simulation	Built-in	External
Learning Curve	Moderate	Steep

2.1.10 Migration Strategies

From Existing OPC UA Infrastructure

Phase 1: Parallel Deployment

- Deploy WaldOT as OPC UA client
- Mirror existing OPC UA data to graph
- Validate data consistency
- No changes to existing systems

Phase 2: Analytics Integration

- Develop Gremlin queries for analytics
- Integrate with BI tools
- Train users on graph queries

Phase 3: Gradual Migration

- Migrate non-critical systems to WaldOT
- Implement rules engine for automation
- Decommission legacy systems incrementally

From SQL Databases

Phase 1: Data Modeling

- Map SQL tables to graph vertices
- Map foreign keys to graph edges
- Identify relationship patterns

Phase 2: ETL Pipeline

- Extract data from SQL
- Transform to graph structure
- Load into WaldOT

Phase 3: Live Sync

- Implement change data capture (CDC)
- Sync SQL changes to graph
- Validate consistency

2.1.11 Roadmap and Future Enhancements

Planned Features

- **Additional Storage Backends**: Cassandra, ArangoDB
- **Enhanced Security**: OAuth2, SAML integration
- **Cloud Connectors**: AWS IoT, Azure IoT Hub, Google Cloud IoT
- **Protocol Bridges**: Modbus, BACnet, MQTT native support
- **Graph Visualization**: Built-in web UI for graph exploration
- **Machine Learning Integration**: TensorFlow, PyTorch model deployment
- **Distributed Computing**: Apache Spark integration for large-scale analytics

Community Contributions Welcome

- Protocol adapters
- Storage backend implementations
- Analytics plugins
- Visualization tools
- Documentation improvements

2.1.12 Decision Framework

When to Use WaldOT

✅ **Good Fit**:

- Complex relationship queries needed
- Multiple data sources to integrate
- Edge computing requirements
- Real-time analytics on industrial data
- Event-driven automation
- Flexible, evolving data models

❌ **Not Ideal**:

- Pure time-series data (use InfluxDB, TimescaleDB)
- Simple SCADA (use traditional SCADA)
- No relationship complexity (use SQL database)
- Extremely high throughput (>100K events/sec sustained)

Evaluation Checklist

- [ ] Do you need complex relationship queries?
- [ ] Do you have multiple industrial protocols to integrate?
- [ ] Do you need edge computing capabilities?
- [ ] Do you need event-driven automation?
- [ ] Do you have a Java 26 runtime available (or can you run the Docker image)?
- [ ] Do you have OPC UA infrastructure or plan to adopt it?
- [ ] Do you need flexible, evolving data models?

If you answered "yes" to 3+ questions, WaldOT is likely a good fit.

2.1.13 Getting Started

Proof of Concept (1 week)

Day 1-2: Installation and basic setup

- Deploy WaldOT Docker container
- Connect OPC UA client (UaExpert)
- Create first graph vertices

Day 3-4: Data simulation and rules

- Configure generator plugin
- Create simple rules
- Test event-driven automation

Day 5: Integration testing

- Connect to existing OPC UA devices
- Develop Gremlin queries
- Evaluate performance

Day 6-7: Documentation and presentation

- Document findings
- Prepare architecture proposal
- Present to stakeholders

Pilot Project (1 month)

Week 1: Infrastructure setup

- Production-grade deployment
- Security configuration
- Monitoring and logging

Week 2-3: Integration

- Connect to production OPC UA devices
- Develop custom plugins if needed
- Implement rules engine logic

Week 4: Validation and handover

- Performance testing
- User training
- Documentation

2.1.14 Conclusion

WaldOT represents a paradigm shift in industrial data management by unifying OPC UA and graph databases. Its plugin architecture, virtual thread concurrency, and edge-first design make it ideal for modern Industrial IoT deployments.

Key Takeaways:

- Unified OT/IT data model reduces integration complexity
- Graph queries enable insights impossible with traditional systems
- Edge computing capabilities reduce latency and bandwidth
- Open standards ensure future-proof architecture
- Plugin system provides unlimited extensibility

Next Steps:

1. Review Developer Guide for hands-on development
2. Review the Configuration Reference and the Docker Compose examples
3. Join the community on GitHub

Questions? Contact: Zona Autonoma srl

3. Users & Operators

3.1 WaldOT User Guide

Operating manual of a WaldOT server with the base plugins (generator, rules engine, TinkerPop server). It explains how the graph is mapped to OPC UA, how to create vertices and edges, how to use the active elements and what to observe while the server runs.

Every example has been executed on the `rossonet/waldot` image. Step-by-step pages are in Examples.

3.1.1 1. Start the server

```
docker run -d --name waldot -p 12686:12686 rossonet/waldot:latest
```

Item	Value
OPC UA endpoint	`opc.tcp://<host>:12686/waldot`
Security	anonymous access enabled, user `admin` / `password123` (change them, see Configuration)
Bootstrap file	`/waldot/boot.conf` (see Bootstrap)
Working directory	`/app`, certificates in `/app/.security`

With a bootstrap file and a fixed security directory:

```
docker run -d --name waldot \
  -p 12686:12686 \
  -v "$PWD/boot.conf:/waldot/boot.conf:ro" \
  -v waldot-security:/app/.security \
  rossonet/waldot:latest \
  --anonymous-access false --factory-password "change-me"
```

Every argument after the image name is a WaldOT option: the complete list is in the Configuration Reference.

The server is ready when the log contains:

```
INFO BOOT - Bootstrap procedure completed successfully.
INFO WaldOT runner - WaldOT OPC UA Server started successfully
```

3.1.2 2. The graph in the OPC UA address space

WaldOT is a TinkerPop graph whose vertices and edges are OPC UA nodes.

```
Objects
├── Gremlin Engine                  root of the graph (ns=2;s=waldot)
│   ├── Vertices
│   │   ├── add vertex                method
│   │   └── <directory>/<subdir>/... folders created by the "directory" property
│   │       └── <vertex>              one object per vertex (ns=2;s=<id>)
│   │           └── <property>        one variable per property
│   ├── Edges
│   └── Variables
└── Commands                        OPC UA methods, grouped by folder
    ├── manage        query, delete directory, edges/...
    ├── information   help, about
    ├── simulator     create generator, create event generator
```

```
├── rules        create rule, create compute, edges/..., showcase/...
└── tinkerpop    create gremlin server
```

Graph element	OPC UA
vertex `id`	NodeId `ns=2;s=<id>` (numeric `ns=2;i=...` when the id is not given)
vertex `label`	browse name and display name (cannot be changed)
`directory` property	folder under `Vertices` (`plant/pumps` → `Vertices/plant/pumps`)
other properties	variables of the vertex object; writable from a client
edge	reference between the two vertex nodes
vertex events	OPC UA events emitted by the vertex object

Any OPC UA client can browse, read, write and subscribe: UaExpert, Prosys Browser, `asyncua` (Python), Eclipse Milo, Node-RED.

3.1.3 3. Vertices and edges

Passive and active elements

Kind	Created with	Behaviour
passive vertex	no `type`	stores properties, exposed as OPC UA variables
active vertex	`type` handled by a plugin	runs logic (generates data, evaluates rules, opens a server)
passive edge	no `type`	relation used by Gremlin traversals
active edge	`type` = `link-to`, `link-from`, `fire`, `rules:execute`	propagates values or events between vertices

Vertex keys

Key	Meaning
`id`	NodeId identifier, unique
`label`	name of the vertex
`type`	active type: `sim:generator`, `sim:event-generator`, `rules:rule`, `rules:compute`, `gremlin`
`directory`	OPC UA folder, levels separated by `/`
any other key	property: string, number or boolean. Use lowercase keys.

Active types of the base distribution

`type`	Plugin	Main properties
`sim:generator`	generator	`algorithm` (`incremental`, `decremental`, `random`, `sinusoidal`, `triangular`, `stopped`), `min`, `max`, `delay` (ms); output in `data`
`sim:event-generator`	generator	`messages` (separated by `|`), `delay`, `event-type`, `algorithm` (`sequential`, `random`, `burst`, `stopped`); runtime `data`, `count`, `severity`
`rules:rule`	rules engine	`condition`, `action` (JEXL), `hysteresis`, `debug`; counters `queue`, `total`, `executed`, `errors`
`rules:compute`	rules engine	`threads`, `execution-timeout-ms`, `factor`; counter `queue`
`gremlin`	TinkerPop	Gremlin Server on `port` (default 1025), `bind` (default `0.0.0.0`)

Complete property tables: generator, rules engine, TinkerPop.

Active edges

Edges are created from the source vertex: `source.addEdge('<label>', target, 'type', '<edge type>', ...)`.

`type`	**Observes**	**Effect**
`link-to`	properties of the **source**	copies the value into the **target**
`link-from`	properties of the **target**	copies the value into the **source** (the source "pulls")
`fire`	properties and events of the **source**	sends the change to the **target** (a rule queues it)
`rules:execute`	(compute → rule)	the compute executes the rule with the given `priority`

Properties of `link-to`, `link-from` and `fire`:

Edge property	**Default**	**Meaning**
`properties-to-monitor`	all	observed properties, separated by `,`; `*` = all
`<property> = <name>`	same name	rename: `'data', 'level'` writes the source `data` into the target `level`
`deadband-value`	none	minimum change to propagate; **must be a number** (`10`, not `'10'`)
`deadband-type`	`absolute`	`absolute` or `percentage` (of the last propagated value)
`delay`	0	the value is written after the given ms
`active`	`true`	`false` disables the edge
`active-property`	`true`	`false` stops property notifications
`active-event`	`true`	`false` stops event notifications (`fire`)
`priority`	10	priority of the notification (`fire`)

3.1.4 4. Bootstrap file

The bootstrap file builds the graph at startup and after a reset. It is a JEXL script: statements end with `;`, comments are `//`.

```
// boot.conf
var gen = graph.addVertex('id', 'level-gen', 'type', 'sim:generator', 'label', 'level-gen',
    'directory', 'plant/sim', 'algorithm', 'sinusoidal', 'min', '0', 'max', '100', 'delay', '200');
var tank = graph.addVertex('id', 'tank-1', 'label', 'tank-1', 'directory', 'plant/tanks', 'level', 0.0);
gen.addEdge('level-signal', tank, 'type', 'link-to', 'properties-to-monitor', 'data', 'data', 'level',
    'deadband-type', 'absolute', 'deadband-value', 10);
log.info('plant ready: ' + cmd.getVerticesCount() + ' vertices');
```

Syntax, detection of the format, available variables and errors: Bootstrap Configuration.

3.1.5 5. Creating elements at runtime

From an OPC UA client: commands

Command	Folder	Input arguments
`query`	`manage`	JEXL expression, e.g. `graph.addVertex('id','v1','label','v1')`
`delete directory`	`manage`	folder NodeId (`v:<directory>`): removes the OPC UA folder, not the vertices
`create generator`	`simulator`	label, directory, algorithm, delay, min, max, post-node-id-data
`create event generator`	`simulator`	label, directory and the event generator properties
`create rule`	`rules`	label, directory, condition, action, debug, hysteresis
`create compute`	`rules`	label, directory, threads, execution-timeout-ms, factor
`create gremlin server`	`tinkerpop`	label, directory, port, bind
`create show case arena` / `evolution` / `gas grid`	`rules/showcase`	see Examples
`about` , `help`	`information`	none

Empty arguments take the default value. Creation commands return the NodeId of the new vertex, or a text starting with `error:` .

Create edges with the `query` command (for example `g.V().has('id','a').next().addEdge('feeds', g.V().has('id','b').next(), 'type', 'link-to')`): the edge commands in `manage/edges` and `rules/edges` do not resolve the NodeId arguments correctly in this version.

Example with Python `asyncua` :

```
import asyncio
from asyncua import Client, ua

async def main():
    async with Client(url="opc.tcp://127.0.0.1:12686/waldot") as c:
        manage = c.get_node("ns=2;s=c:manage")
        query = ua.NodeId.from_string("ns=2;s=query")
        result = await manage.call_method(query, ua.Variant(
            "graph.addVertex('id','from-opcua','label','from-opcua','directory','opcua')",
            ua.VariantType.String))
        print(result)            # ['v[ns=2;s=from-opcua]', '']
        about = await c.get_node("ns=2;s=c:information").call_method(
            ua.NodeId.from_string("ns=2;s=about"))
        print(about)             # name, description, license, ..., commit, version, branch

asyncio.run(main())
```

From the bootstrap or the `query` command

All commands are available in JEXL as `<folder>_<name>` objects with an `exec(...)` method:

```
var r = simulator_create_generator.exec('cmd-gen', 'plant/sensors', 'incremental', '1000', '0', '10', '');
log.info('created ' + r[0]);
```

From a Gremlin client

Create a `gremlin` vertex (or run `create gremlin server`): a TinkerPop Gremlin Server exposes the same graph.

```
tinkerpop_create_gremlin_server.exec('gremlin-1', 'services', '8182', '0.0.0.0');
```

```
docker run -d -p 12686:12686 -p 8182:8182 -v "$PWD/boot.conf:/waldot/boot.conf:ro" rossonet/waldot:latest
curl -X POST -d '{"gremlin":"g.V().count()"}' http://127.0.0.1:8182
```

The Gremlin Server has no authentication: publish its port only on trusted networks.

3.1.6 6. Rules engine

```
[source vertex] --fire--> [rules:rule] <--rules:execute-- [rules:compute] --> virtual threads
```

1. A property of the source vertex changes; the `fire` edge notifies the rule.
2. The rule queues the notification (`queue`); notifications from the same source within `hysteresis` ms are discarded.
3. The compute takes the rules with queued events, highest weight first: `weight = priority × factor + queue` .
4. At most `threads` rules run at the same time; each execution evaluates `condition` and, when `true` , runs `action` .

Variables in `condition` and `action` : `g` , `graph` , `log` , `cmd` , `self` (the rule vertex), `math` , `rand` and all commands. Each execution has its own context: keep state in vertex properties (`self.property('n', 1)`).

```
var compute = graph.addVertex('id', 'plant-compute', 'type', 'rules:compute', 'label', 'plant-compute',
    'directory', 'plant/rules', 'threads', '2');
var high = graph.addVertex('id', 'tank-high', 'type', 'rules:rule', 'label', 'tank-high', 'directory', 'plant/rules',
    'condition', "g.V().has('id','tank-1').values('level').next() > 80",
    'action', "g.V().has('id','ctrl-1').next().property('output', 0.0); log.warn('tank-1 high level, valve closed')",
    'hysteresis', '1000');
compute.addEdge('runner', high, 'type', 'rules:execute', 'priority', '100');
tank.addEdge('watch-level', high, 'type', 'fire', 'properties-to-monitor', 'level');
```

Rule counters

Variable	Meaning	What to check
`queue`	notifications waiting	grows continuously → compute too slow (raise `threads`) or hysteresis too low
`total`	evaluations	does not grow → the `fire` edge or the `rules:execute` edge is missing
`executed`	actions completed	stays at 0 with `total` growing → condition always false
`errors`	compilation or runtime errors	grows → read the log, set `debug` to 2

`condition` and `action` can be changed at runtime from a client: the script is compiled again. With `debug` = 1 every phase is published as an OPC UA event, with `debug` = 2 it is also written in the log.

3.1.7 7. Monitoring

What	Where
values of the plant	variables of the vertices, OPC UA subscription
simulated alarms	events of `sim:event-generator` vertices (subscribe to events of the vertex or of the server)
rules health	`queue` , `total` , `executed` , `errors` of each rule; `queue` of each compute
software version	`about` command (name, license, git commit, version, branch)
server log	`docker logs waldot` ; categories `BOOT` , `CONSOLE` , `WaldOT runner`

Useful queries (`query` command):

```
g.V().count().next()
g.V().has('type', 'rules:rule').values('label').toList()
g.V().has('id', 'tank-high').values('errors').next()
g.V().has('id', 'tank-high').inE().toList()
cmd.listCommands()
```

3.1.8 8. Operations

Task	How
change the graph at startup	edit `boot.conf`, restart the container
rebuild the graph	restart the container
remove a group of vertices	`query` with a loop, see below
run the bootstrap again	`cmd.reset()`: plugins are restarted and the bootstrap runs again, but the existing vertices are **not** removed
keep the OPC UA certificate	mount a volume on `/app/.security`
restrict access	`--anonymous-access false`, `--factory-password ...`
disable a command	`--<cmd>-command-executable false` (Configuration)

Remove all the vertices whose id starts with `arena-`:

```
var n = 0; for (var v : g.V().toList()) { if (v.id().getIdentifier().toString().startsWith('arena-')) { v.remove(); n = n + 1; } } n;
```

The graph lives in memory: changes made at runtime are lost at restart unless they are also in the bootstrap file.

3.1.9 9. Troubleshooting

Symptom	Cause	Fix
`ambiguous statement error` at boot	missing `;` or Groovy syntax in the bootstrap	see Bootstrap Configuration
the target of a `link-to` does not change	wrong `properties-to-monitor`, or deadband too large	check the property names; `deadband-value` must be a number
deadband has no effect	`deadband-value` passed as a string	use `'deadband-value', 10`
rule `total` stays at 0	missing `fire` or `rules:execute` edge	check edges with `g.V().has('id','<rule>').inE().toList()`
rule `errors` grows	JEXL error in condition/action	set `debug` to 2 and read the log
`Vertex with id already exists`	duplicated id	use unique ids
`already exists` from a showcase command	the scenario is already in the graph	remove its vertices (see Operations) or restart
client cannot connect	port not published, or wrong host in the endpoint	publish `-p 12686:12686`, set `--bind-hostname`

3.1.10 Related documentation

- Examples
- Configuration Reference
- Bootstrap Configuration
- Architecture Overview
- Integration Guide - writing plugins

3.2 WaldOT Configuration Reference

This page documents every runtime parameter accepted by the WaldOT server (`waldot-app`). The source of truth is `waldot-runner/src/main/java/net/rossonet/agent/AbstractWaldotRunner.java` : all options are declared there with Picocli and inherited by `net.rossonet.agent.WaldotRunner` .

3.2.1 How configuration is applied

WaldOT reads its configuration from three sources:

Source	Example	Notes
Command-line options	`--tcp-port 4840`	Primary mechanism, parsed by Picocli.
Java system properties	`-Dwaldot.milo.strategy=base`	Low-level switches of the namespace module.
Bootstrap file	`--boot-url file:///waldot/boot.conf`	Initial graph content (see Bootstrap).

Picocli also supports `@file` argument files (`showAtFileInUsageHelp` is enabled), so a long option list can be stored in a file and passed as `@/path/to/waldot.args` .

Environment variables are **not** used to configure WaldOT: every runtime parameter is a command-line option. In Docker, options are simply appended after the image name (see Docker usage).

Boolean options (`--anonymous-access` and all `*-executable` / `*-user-executable` flags) accept an optional value: `--anonymous-access false` disables the feature, the option alone (`--anonymous-access`) means `true` .

To print the full, always up-to-date option list:

```
# Docker image
docker run --rm rossonet/waldot:latest --help
# fat jar built with ./gradlew waldot-app:shadowJar
java -jar waldot-app/build/libs/waldot-app-<version>-all.jar --help
```

3.2.2 Network options

Option	Short	Default	Description
`--tcp-port`	`-tp`	`12686`	TCP port of the OPC UA binary endpoint (`opc.tcp://`).
`--https-port`	`-hp`	`8443`	HTTPS port for OPC UA web services.
`--bind-addresses`	`-ba`	`0.0.0.0`	Comma-separated list of bind addresses.
`--bind-hostname`	`-bh`	`127.0.0.1`	Hostname published in endpoint URLs and certificates.
`--endpoint-path`	`-ep`	`/waldot`	Path component of the endpoint URL.

The resulting endpoint is `opc.tcp://<bind-hostname>:<tcp-port><endpoint-path>` , for example `opc.tcp://127.0.0.1:12686/waldot` .

3.2.3 Server identity

Option	Short	Default	Description
`--application-name`	`-an`	`WaldOT OPCUA server`	OPC UA application name.
`--product-name`	`-pn`	`WaldOT`	Product name in `BuildInfo`.
`--product-uri`	`-pu`	`urn:rossonet:waldot:uaserver`	Product URI in the application description.
`--manufacturer-name`	`-mn`	`Zona Autonoma srl`	Manufacturer name in `BuildInfo`.
`--build-number`	`-bn`	`w001`	Build number in `BuildInfo`.
`--namespace-uri`	`-nu`	`urn:rossonet:waldot:engine`	URI of the WaldOT namespace.

3.2.4 Security and authentication

Option	Short	Default	Description
`--security-dir`	`-sd`	`.security`	Directory for certificates, private keys and PKI trust lists (relative to the working directory, `/app` in Docker).
`--dns-address-cert`	`-dc`	`127.0.0.1`	DNS name used as Subject Alternative Name of the generated certificate.
`--anonymous-access`	`-aa`	`true`	Allow anonymous OPC UA sessions.
`--factory-username`	`-fu`	`admin`	Initial administrator user name.
`--factory-password`	`-fp`	`password123`	Initial administrator password. **Change it in production.**

Production hardening

With the default values anonymous access is enabled, the administrator password is public and the `exec` command (which runs operating-system commands) is enabled. Always override `--anonymous-access`, `--factory-password` and `--exec-command-executable` outside development.

Known limitation

In the current release `--anonymous-access false` is parsed and stored in the configuration, but the anonymous identity validators still accept anonymous sessions. Do not rely on this flag alone to protect a server exposed on an untrusted network.

3.2.5 Application settings

Option	Short	Default	Description
`--boot-url`	`-bu`	`file:///waldot/boot.conf`	URL of the bootstrap file (`file://`, `http://`, `https://`).
`--help-directory`	`-hd`	`/app/help`	Directory with the help files exposed by the `help` command.
`--facts-valid-delay`		`0`	Default delay (ms) before a fact becomes valid in the rules engine.
`--facts-valid-until`		`0`	Default validity (ms) of a fact in the rules engine (`0` = never expires).

3.2.6 Address-space root nodes

Option	Default	Description
`--root-node-id`	`waldot`	NodeId of the WaldOT root folder.
`--root-node-browse-name`	`Gremlin Engine`	Browse name of the root folder.
`--root-node-display-name`	`Gremlin Engine`	Display name of the root folder.
`--asset-node-id`	`aas`	NodeId of the asset administration folder.
`--asset-node-browse-name`	`Administration`	Browse name of the asset folder.
`--asset-node-display-name`	`Administration`	Display name of the asset folder.
`--interface-node-id`	`cmd`	NodeId of the commands folder.
`--interface-node-browse-name`	`Commands`	Browse name of the commands folder.
`--interface-node-display-name`	`Commands`	Display name of the commands folder.

3.2.7 Built-in OPC UA commands

Each built-in command is an OPC UA method under the commands folder and is configured by four options with the same pattern: `--<cmd>-command-label`, `--<cmd>-command-description`, `--<cmd>-command-executable`, `--<cmd>-command-user-executable`.

Command (`<cmd>`)	Default label	Default description	Purpose
`about`	`about`	`info about this software`	Software information.
`help`	`help`	`list available commands`	Lists available commands / help files.
`waldot`	`query`	`run Gremlin query`	Executes a JEXL expression (with `g`, `graph`, `log`, `cmd`) on the graph.
`exec`	`exec`	`run system command`	Executes an operating-system command.

All `*-executable` and `*-user-executable` flags default to `true`.

Example: disable the `exec` command entirely

```
--exec-command-executable false --exec-command-user-executable false
```

3.2.8 Java system properties

Property	Default	Description
`waldot.milo.strategy`	`optimized`	Milo strategy used by the namespace: `optimized` or `base` (legacy). Unknown values fall back to `optimized`.
`waldot.boot.file`	`/tmp/boot.conf`	Bootstrap file used by `OpcFactory.getOpcGraph()` (embedded/test usage only, not by the server runner).

3.2.9 Bootstrap file

The bootstrap file is loaded at startup from `--boot-url`. Its format is detected automatically:

- **script mode** (recommended): the whole file is one JEXL script with variables, functions, loops and `//` comments; every statement must end with `;`;
- **line mode** (legacy): one expression per line, `#` comments.

The file is executed again by `cmd.reset()`, which does not remove the existing vertices.

The detailed syntax and detection rules are maintained in `waldot-namespace/BOOTSTRAP_CONFIGURATION.md`.

3.2.10 Docker usage

The image is built from the repository `Dockerfile` (multi-stage build: `eclipse-temurin:26-jdk` to compile, `eclipse-temurin:26-jre-alpine` at runtime):

```
docker buildx build --load -t rossonet/waldot:latest .
```

Image layout:

Item	Value
Working directory	`/app`
Libraries	`/app/lib` (waldot-app, generator, rules engine and TinkerPop plugins, dependencies)
Help files	`/app/help` (`index.txt`, `author.txt`, `license.txt`)
Security directory	`/app/.security` (declared as volume)
Bootstrap file	`/waldot/boot.conf` (mount it, default of `--boot-url`)
Exposed ports	`12686` (OPC UA TCP), `8443` (HTTPS)
User	`waldot` (uid/gid `1000`, non-root)
Entrypoint	`java --add-opens ... -cp /app/lib/* net.rossonet.agent.MainAgent`
Default command	empty

Every argument after the image name is passed to WaldOT as an option:

```
docker run -p 4840:4840 \
  -v "$PWD/boot.conf:/waldot/boot.conf:ro" \
  rossonet/waldot:latest \
  --tcp-port 4840 \
  --anonymous-access false \
  --factory-password 'S3cure!' \
  --exec-command-executable false
```

The same applies to `command:` in Docker Compose:

```
services:
  waldot:
    image: rossonet/waldot:latest
    ports:
      - "4840:4840"
    volumes:
      - waldot-security:/app/.security
      - ./boot.conf:/waldot/boot.conf:ro
    command:
      - "--tcp-port"
      - "4840"
volumes:
  waldot-security:
```

JVM options (heap size, system properties) are supplied through the standard `JDK_JAVA_OPTIONS` variable, read by the `java` launcher itself:

```
docker run -e JDK_JAVA_OPTIONS="-Xmx2g -Dwaldot.milo.strategy=optimized" rossonet/waldot:latest
```

If no file is mounted on `/waldot/boot.conf` the server starts with an empty graph and logs `the bootstrap configuration is empty or null`.

Host directories mounted for writing (for example a bind mount on `/app/.security`) must be writable by uid `1000`.

Ready-to-use Compose files are available in `waldot-app/examples`.

3.2.11 Exit codes

`net.rossonet.agent.MainAgent` terminates the JVM with the code returned by Picocli: `0` on normal shutdown, `1` for an exception during execution, `2` for invalid command-line arguments.

3.3 Examples

3.3.1 Examples

Step-by-step examples for a WaldOT server with the base plugins. Each page can be run on the Docker image: save the script as `boot.conf` and start

```
docker run --rm --name waldot -p 12686:12686 \
  -v "$PWD/boot.conf:/waldot/boot.conf:ro" rossonet/waldot:latest
```

then open `opc.tcp://localhost:12686/waldot` with an OPC UA client and browse `Gremlin Engine/Vertices`.

Page	Topics
1. Plain vertices	vertices, properties, directories, passive edges, Gremlin traversals
2. Active edges	`link-to`, rename, deadband, delay, `link-from`
3. Generators and rules	`sim:generator`, `sim:event-generator`, `rules:rule`, `rules:compute`, `fire`, `rules:execute`
4. Complete bootstrap	the previous pages in one file, plugin commands, Gremlin Server
5. Showcase: Dungeon Arena	rules with priorities, hysteresis, threads, errors
6. Showcase: Evolution Lab	parallel rules, evolutionary algorithm, more than 100 vertices
7. Showcase: Gas Grid	PI control loops, deadband and delay on links, safety rules

Scripts use the JEXL syntax described in Bootstrap Configuration: statements end with `;`.

3.3.2 1. Plain vertices

A plain (passive) vertex has no `type`: it only stores properties, which are published as OPC UA variables and can be read and written by any client.

Script

```
// 01 - plain vertices
var pump = graph.addVertex('id', 'pump-1', 'label', 'pump-1', 'directory', 'plant/pumps',
    'manufacturer', 'ACME', 'power-kw', 7.5, 'running', true);
var tank = graph.addVertex('id', 'tank-1', 'label', 'tank-1', 'directory', 'plant/tanks', 'level', 0.0);

// passive edge: a relation for traversals, no data flow
pump.addEdge('fills', tank);

log.info('01 out of pump: ' + g.V().has('id', 'pump-1').out('fills').values('label').next());
```

Log:

```
INFO CONSOLE - 01 out of pump: tank-1
```

Result in OPC UA

```
Gremlin Engine/Vertices
└── plant
    ├── pumps
    │   └── pump-1          ns=2;s=pump-1
    │       ├── manufacturer   "ACME"   (String)
    │       ├── power-kw       7.5      (Double)
    │       └── running        true     (Boolean)
    └── tanks
        └── tank-1          ns=2;s=tank-1
            └── level          0.0      (Double)
```

- `id` becomes the NodeId, `label` the browse name.
- `directory` creates the folders `plant/pumps` and `plant/tanks`.
- The value keeps the JEXL type: `'7.5'` is a String, `7.5` a Double, `true` a Boolean. The variables are declared as `BaseDataType`, so a client can write any type.

Try

Action	Result
write `running` = `false` from the client	the graph property changes: `g.V().has('id','pump-1').values('running').next()` returns `false`
`query`: `g.V().has('id','tank-1').next().property('level', 12.5)`	the OPC UA variable `level` shows 12.5
`query`: `g.V().has('id','tank-1').in('fills').values('label').toList()`	`[pump-1]`

Next: 2. Active edges.

3.3.3 2. Active edges

Active edges copy values between vertices when a property changes. They need the vertices of page 1.

Edge	Observes	Writes
`link-to`	source	target
`link-from`	target	source

link-to with rename and deadband

```
// 02 - generator -> tank level, only changes larger than 10
var gen = graph.addVertex('id', 'level-gen', 'type', 'sim:generator', 'label', 'level-gen', 'directory', 'plant/sim',
    'algorithm', 'sinusoidal', 'min', '0', 'max', '100', 'delay', '200');
gen.addEdge('level-signal', tank, 'type', 'link-to', 'properties-to-monitor', 'data', 'data', 'level',
    'deadband-type', 'absolute', 'deadband-value', 10);
```

- `properties-to-monitor` = `data`: only `data` of the generator is observed.
- `'data', 'level'`: the value is written in `level` of the tank (rename).
- `deadband-value` = 10: a value is propagated only when it differs by more than 10 from the last value of `level`. It must be a number, not a string.

Observed values (generator every 200 ms):

`level-gen.data`	`tank-1.level`
27.80	27.80
62.69	62.69
96.00	91.34 (change smaller than 10, not propagated)

With `deadband-type` = `percentage` the threshold is a percentage of the last value.

link-to with delay

```
var mirror = graph.addVertex('id', 'tank-1-hmi', 'label', 'tank-1-hmi', 'directory', 'plant/hmi', 'level', 0.0);
tank.addEdge('to-hmi', mirror, 'type', 'link-to', 'properties-to-monitor', 'level', 'delay', 2000);
```

`tank-1-hmi.level` follows `tank-1.level` 2 seconds later: use it to model transport delays (pipes, networks, slow transmitters).

link-from

```
// the valve pulls the command of the controller
var ctrl = graph.addVertex('id', 'ctrl-1', 'label', 'ctrl-1', 'directory', 'plant/ctrl', 'output', 0.0);
var valve = graph.addVertex('id', 'valve-1', 'label', 'valve-1', 'directory', 'plant/valves', 'opening', 0.0);
valve.addEdge('command', ctrl, 'type', 'link-from', 'properties-to-monitor', 'output', 'output', 'opening');
ctrl.property('output', 42.0);
log.info('02 valve opening after link-from: ' + g.V().has('id', 'valve-1').values('opening').next());
```

```
INFO CONSOLE - 02 valve opening after link-from: 42.0
```

The edge goes from the valve to the controller, but the data flows from the controller (`output`) to the valve (`opening`). Use `link-from` when the consumer owns the relation (an actuator reading its setpoint).

Edge properties

Property	Default	Meaning
`properties-to-monitor`	all	observed properties, separated by `,` ; `*` = all
`<source property>`	same name	name of the destination property
`deadband-value`	none	minimum change (number)
`deadband-type`	`absolute`	`absolute` or `percentage`
`delay`	0	ms before writing
`active`	`true`	`false` disables the edge
`active-property`	`true`	`false` stops property notifications

All of them can be changed at runtime, for example from the `query` command: `g.E().hasLabel('to-hmi').next().property('delay', 500)`.

Next: 3. Generators and rules.

3.3.4 3. Generators and rules

This page adds active vertices to the plant of pages 1 and 2: a rule that closes the valve when the tank is too full, and a simulated alarm source.

Rule

```
// 03 - compute + rule + edges
var compute = graph.addVertex('id', 'plant-compute', 'type', 'rules:compute', 'label', 'plant-compute',
    'directory', 'plant/rules', 'threads', '2');
var high = graph.addVertex('id', 'tank-high', 'type', 'rules:rule', 'label', 'tank-high', 'directory', 'plant/rules',
    'condition', "g.V().has('id','tank-1').values('level').next() > 80",
    'action', "g.V().has('id','ctrl-1').next().property('output', 0.0); log.warn('tank-1 high level, valve closed')",
    'hysteresis', '1000');

// the compute executes the rule
compute.addEdge('runner', high, 'type', 'rules:execute', 'priority', '100');
// every change of tank-1.level notifies the rule
tank.addEdge('watch-level', high, 'type', 'fire', 'properties-to-monitor', 'level');
```

```
level-gen --link-to--> tank-1 --fire--> tank-high <--rules:execute-- plant-compute
                                          | action
                                          v
                                  ctrl-1.output = 0 --link-from--> valve-1.opening = 0
```

Log when the level exceeds 80 (the `log` of a rule writes with the rules engine logger, in the thread of the rule):

```
[ns=2;s=tank-high] WARN net.rossonet.waldot.rules.WaldotRulesEnginePlugin - tank-1 high level, valve closed
```

Counters of `tank-high` after about 20 seconds:

queue	total	executed	errors
0	50	7	0

- `total` counts the evaluations (one per notification not discarded by the 1 s hysteresis);
- `executed` counts the evaluations with a true condition;
- `errors` stays at 0.

TRY

Action	Result
write `condition` = `false` on `tank-high`	`executed` stops growing; the script is compiled again
write `debug` = 2	every phase of the rule is published as an OPC UA event and logged
write `threads` = 4 on `plant-compute`	up to 4 rules of the compute run at the same time
write an invalid condition (`g.V(`)	`errors` grows

Event generator

```
var ev = graph.addVertex('id', 'alarm-sim', 'type', 'sim:event-generator', 'label', 'alarm-sim', 'directory', 'plant/sim',
    'messages', 'pump overload|valve stuck|low pressure', 'delay', '3000', 'event-type', 'device-failure');
```

Every 3 seconds `alarm-sim` publishes an OPC UA event of type `DeviceFailureEventType` with the next message. Subscribe to the events of the vertex with the client. The variables show the last event:

Variable	Example
`data`	`low pressure`
`count`	`18`
`severity`	`351`

Events can be sent to other vertices with a `fire` edge, for example to a rule:

```
ev.addEdge('alarms', someRule, 'type', 'fire');
```

Generator algorithms

`algorithm`	Values
`incremental` / `decremental`	from `min` to `max` (or back)
`random`	random between `min` and `max`
`sinusoidal` / `triangular`	wave between `min` and `max`
`stopped`	the last value stays

`algorithm`, `min`, `max` and `delay` can be changed at runtime.

Next: 4. Complete bootstrap.

3.3.5 4. Complete bootstrap

The plant of pages 1-3 in one bootstrap file, plus a Gremlin Server opened by a plugin command. This file has been executed as it is.

boot.conf

```
// --- 01 plain vertex
var pump = graph.addVertex('id', 'pump-1', 'label', 'pump-1', 'directory', 'plant/pumps',
    'manufacturer', 'ACME', 'power-kw', 7.5, 'running', true);
var tank = graph.addVertex('id', 'tank-1', 'label', 'tank-1', 'directory', 'plant/tanks', 'level', 0.0);
pump.addEdge('fills', tank);
log.info('01 out of pump: ' + g.V().has('id', 'pump-1').out('fills').values('label').next());

// --- 02 link-to with rename, deadband, delay
var gen = graph.addVertex('id', 'level-gen', 'type', 'sim:generator', 'label', 'level-gen', 'directory', 'plant/sim',
    'algorithm', 'sinusoidal', 'min', '0', 'max', '100', 'delay', '200');
gen.addEdge('level-signal', tank, 'type', 'link-to', 'properties-to-monitor', 'data', 'data', 'level',
    'deadband-type', 'absolute', 'deadband-value', 10);
var mirror = graph.addVertex('id', 'tank-1-hmi', 'label', 'tank-1-hmi', 'directory', 'plant/hmi', 'level', 0.0);
tank.addEdge('to-hmi', mirror, 'type', 'link-to', 'properties-to-monitor', 'level', 'delay', 2000);

// --- 02 link-from: the valve pulls the command from the controller
var ctrl = graph.addVertex('id', 'ctrl-1', 'label', 'ctrl-1', 'directory', 'plant/ctrl', 'output', 0.0);
var valve = graph.addVertex('id', 'valve-1', 'label', 'valve-1', 'directory', 'plant/valves', 'opening', 0.0);
valve.addEdge('command', ctrl, 'type', 'link-from', 'properties-to-monitor', 'output', 'output', 'opening');
ctrl.property('output', 42.0);
log.info('02 valve opening after link-from: ' + g.V().has('id', 'valve-1').values('opening').next());

// --- 03 rule
var compute = graph.addVertex('id', 'plant-compute', 'type', 'rules:compute', 'label', 'plant-compute',
    'directory', 'plant/rules', 'threads', '2');
var high = graph.addVertex('id', 'tank-high', 'type', 'rules:rule', 'label', 'tank-high', 'directory', 'plant/rules',
    'condition', "g.V().has('id','tank-1').values('level').next() > 80",
    'action', "g.V().has('id','ctrl-1').next().property('output', 0.0); log.warn('tank-1 high level, valve closed')",
    'hysteresis', '1000');
compute.addEdge('runner', high, 'type', 'rules:execute', 'priority', '100');
tank.addEdge('watch-level', high, 'type', 'fire', 'properties-to-monitor', 'level');

// --- 03 event generator
var ev = graph.addVertex('id', 'alarm-sim', 'type', 'sim:event-generator', 'label', 'alarm-sim', 'directory', 'plant/sim',
    'messages', 'pump overload|valve stuck|low pressure', 'delay', '3000', 'event-type', 'device-failure');

// --- tinkerpop: label, directory, port, bind
var gs = tinkerpop_create_gremlin_server.exec('gremlin-1', 'services', '8182', '0.0.0.0');
log.info('gremlin server: ' + gs[0]);
```

Run

```
docker run --rm --name waldot -p 12686:12686 -p 8182:8182 \
  -v "$PWD/boot.conf:/waldot/boot.conf:ro" rossonet/waldot:latest
```

Log:

```
INFO BOOT - Bootstrap configuration loaded in SCRIPT MODE from URL: [file:///waldot/boot.conf]
INFO CONSOLE - 01 out of pump: tank-1
INFO CONSOLE - 02 valve opening after link-from: 42.0
INFO CONSOLE - gremlin server: ns=2;i=120001
INFO BOOT - Bootstrap procedure completed successfully.
```

Query the graph with Gremlin

```
curl -X POST -d '{"gremlin":"g.V().values(\"label\").fold()"}' http://127.0.0.1:8182
```

```
... "@value":["tank-1","tank-1-hmi","tank-high","alarm-sim","level-gen","pump-1","ctrl-1","plant-compute","valve-1","gremlin-1"] ...
```

The Gremlin Server works on the same graph published in OPC UA: a vertex added with Gremlin appears in the address space.

Notes

- Plugin commands are available as `<folder>_<command name>` with spaces replaced by `_`; `exec(...)` takes the OPC UA input arguments as strings.
- An error in the script stops it: the vertices created before the error remain. Use `try { ... } catch (var e) { ... }` for optional parts.

- `cmd.reset()` runs the file again without removing the existing vertices: creating the same ids fails with `Vertex with id already exists`.

Next: 5. Showcase: Dungeon Arena.

3.3.6 5. Showcase: Dungeon Arena

A small tower-defense game made only of generators, edges and rules. It shows priorities, two computes with a different number of threads, hysteresis, debug and a rule that always fails.

Create

From an OPC UA client: `Commands/rules/showcase/create` `show case arena`. From a bootstrap or the `query` command:

```
log.info(create_show_case_arena.exec('')[0]);
```

```
show case arena created: 3 generators, 4 state vertices, 2 compute, 7 rules
```

Argument	Default	Meaning
`delay-ms`	250 (minimum 10)	period of the generators; the turn clock uses twice this value

A second call answers `show case arena already exists: remove its vertices before creating it again`.

Graph (16 vertices, folder `rules/showcase/arena`)

Folder	Vertices
`state`	`arena-tower` (`data` = mana, `hp`, `kills`, `spells`), `arena-wave` (`data` = enemies, `difficulty`), `arena-player` (`level`, `gold`, `score`, `defeats`), `arena-turn` (`data`)
`generators`	`arena-gen-wave` (random 0-100), `arena-gen-mana` (sinusoidal 0-100), `arena-gen-turn` (incremental); `link-to` edges into the state vertices
`compute`	`arena-combat` (4 threads), `arena-economy` (1 thread)
`rules`	7 rules, below
`triggers`	`fire` edges

Rule	Fired by	Condition	Action	Compute (priority)
`arena-rule-fire-spell`	`tower.data`	mana > 80 and enemies > 20	`kills` +1, `spells` +1; hysteresis 300 ms	combat (200)
`arena-rule-tower-damage`	`wave.data`	enemies > 50 + 5 × level	`hp` decreases	combat (150)
`arena-rule-spawn-wave`	`wave.data`	enemies > 70 and difficulty < 10	`difficulty` +1	combat (100)
`arena-rule-broken`	`turn.data`	`1 / 0 > 0` (always fails)	never executed	combat (10)
`arena-rule-game-over`	`tower.hp`	hp ≤ 0	`defeats` +1, the game restarts	economy (200)
`arena-rule-level-up`	`tower.kills`	kills multiple of 5	`level` +1, `gold` += 10 × level; `debug` = 1	economy (100)
`arena-rule-turn-tick`	`turn.data`	turn multiple of 10	`score` += 10 × level	economy (50)

What to observe

Values after about one minute:

Vertex	Values
`arena-tower`	hp 72, kills 3, spells 31
`arena-wave`	difficulty 7
`arena-player`	level 1, gold 110, score 170, defeats 4

Rule	`total`	`executed`	`errors`
`arena-rule-spawn-wave`	255	42	0
`arena-rule-tower-damage`	255	100	0
`arena-rule-fire-spell`	128	31	0
`arena-rule-game-over`	104	4	0
`arena-rule-turn-tick`	127	13	0
`arena-rule-level-up`	35	5	0
`arena-rule-broken`	127	0	127

- `total` > `executed`: most notifications have a false condition.
- `fire-spell` has half the evaluations of `spawn-wave` although both follow a generator with the same period: the 300 ms hysteresis discards notifications.
- `arena-rule-broken` shows how a failing rule looks: `errors` = `total`, `executed` = 0, a JEXL exception in the log.
- `arena-rule-level-up` (`debug` = 1) publishes an OPC UA event for each phase: subscribe to the events of the vertex.

Try

Action	Effect
`arena-gen-wave.algorithm` = `stopped`	no new waves, the tower recovers
`arena-economy.threads` = 4	economy rules run in parallel
`arena-rule-broken.condition` = `false`	`errors` stops growing
`arena-rule-fire-spell.hysteresis` = 0	more spells, `total` grows faster
priority of the edge `runs-broken` = 300	the broken rule is served first in `arena-combat`

Next: 6. Showcase: Evolution Lab.

3.3.7 6. Showcase: Evolution Lab

An evolutionary algorithm written as rules. Each individual of the population is a vertex; one rule per individual mutates it in parallel; a second compute with one thread runs selection, crossover and statistics. The goal is to maximise `100 - Rastrigin(x, y, z)` (maximum 100 at 0, 0, 0).

Create

From an OPC UA client: `Commands/rules/showcase/create show case evolution`. From a bootstrap or the `query` command:

```
log.info(create_show_case_evolution.exec('', '', '', '')[0]);
```

```
show case evolution created: 40 individuals, 42 generators, 2 compute, 46 rules, 4 control vertices
```

Argument	Default	Range	Meaning
`population-size`	40	30-1000	individuals (vertices = 3 × N + 14)
`delay-ms`	200	≥ 10	period of the generators
`target-fitness`	99		fitness that stops the evolution
`seed`	42		seed of the initial population

Graph (folder `rules/showcase/evolution`)

Folder	Content
`population`	`evo-ind-0` … `evo-ind-N`: genes `x`, `y`, `z`, `fitness`, `data` (noise)
`generators`	one `random` generator per individual (mutation noise, `link-to` into `data`), `evo-gen-clock` (generations), `evo-gen-rate` (mutation rate 5-50)
`rules/evaluate`	`evo-rule-evaluate-i`: when the noise is below the mutation rate, mutates the genes and keeps the mutation only if the fitness improves
`rules/breeding`	`evo-rule-select-best`, `evo-rule-crossover`, `evo-rule-stagnation`, `evo-rule-converged`, `evo-rule-stats` (`debug` = 1), `evo-rule-bad-input` (fails on purpose)
`compute`	`evo-eval-pool` (8 threads, evaluation rules), `evo-breeding` (1 thread, breeding rules)
`control`	`evo-hall-of-fame`, `evo-generation`, `evo-mutation-rate`, `evo-stats`

Breeding priorities: converged 190, crossover 150, stagnation 100, stats 80, bad-input 60, select-best 50. `select-best` is the most frequent rule, so it has a low priority: with one thread the others must not starve.

What to observe

Values after about one minute (40 individuals):

Vertex	Values
`evo-hall-of-fame`	fitness 97.8 at x 1.0, y 0.96, z 0.01; improvements 11; stagnations 4; status `running`
`evo-stats`	mean 91.46, best 97.8, worst 83.13
`evo-mutation-rate`	data 21.6, boost 30

Rule	`queue`	`total`	`executed`	`errors`
`evo-rule-select-best`	1	281	281	0
`evo-rule-crossover`	0	64	22	0
`evo-rule-stagnation`	0	64	5	0
`evo-rule-stats`	0	64	64	0
`evo-rule-converged`	0	11	0	0
`evo-rule-bad-input`	0	64	0	64
`evo-rule-evaluate-0`	0	320	122	0

- `evo-hall-of-fame.fitness` grows over time; `status` becomes `converged` when it reaches `target-fitness` and the generators stop.
- `stagnations` and `evo-mutation-rate.boost` grow when the best does not improve: the mutation rate increases to escape local maxima (here the algorithm is near the local maximum at $x \approx 1$, $y \approx 1$).
- `queue` of `evo-rule-select-best` is often 1: the single thread of `evo-breeding` is busy.

Try

Action	Effect
`evo-eval-pool.threads` = 1	evaluations are serialized, `queue` of the evaluate rules grows
`evo-breeding.threads` = 4	breeding rules run in parallel
`evo-gen-rate.algorithm` = `stopped`	the mutation rate is fixed
`evo-rule-converged.condition` = `true`	at its next notification the rule sets `status` = `converged` and stops all generators

Next: 7. Showcase: Gas Grid.

3.3.8 7. Showcase: Gas Grid

A gas distribution network controlled by **PI loop controllers** written as rules. It is designed for demos to gas utilities: plants, transmitters, valves and controllers are vertices; measurements travel on `link-to` / `link-from` edges with deadband and delay; control and safety logic are rules.

The model is didactic, **not a hydraulic simulation**: the process is a first-order system and all values (pressures, demands, thresholds, delays) are indicative.

Create

From an OPC UA client: `Commands/rules/showcase/create show case gas grid`. From a bootstrap or the `query` command:

```
log.info(create_show_case_gas_grid.exec('', '', '', '', '', '')[0]);
```

```
show case gas grid created: 4 stations, gas metano, 58 vertices
```

Argument	Default	Range	Meaning
`stations`	4	2-20	pressure reduction stations
`delay-ms`	200	≥ 10	simulation step
`setpoint`	25	5-100	downstream pressure (mbar)
`pipe-delay-ms`	300	0-10000	dead time of the pipe
`gas-type`	`metano`	`metano`, `metano-blu`	gas profile
`compare`	`true`	`true`, `false`	the last station has no deadband, for comparison

Invalid arguments create nothing (`argument 'stations' must be between 2 and 20`). Vertices: `14 + 11 × stations`.

GAS PROFILES (INDICATIVE VALUES)

Parameter	`metano`	`metano-blu` **(20 % hydrogen)**
relative density	0.60	0.48
flow factor at the same opening	1.00	1.12
nominal demand	50	57.5
ESD threshold over the setpoint	+30 %	+25 %
transmitter deadband	2.0 mbar	1.2 mbar
demand deadband	5 %	3 %
upstream pressure deadband	0.3 bar	0.2 bar
dead time	`pipe-delay-ms` × 1.0	`pipe-delay-ms` × 1.2

The profile is stored on the `gas-network` vertex.

Graph (folder `rules/showcase/gas-grid`)

```
supply/         gas-gate-gen (upstream pressure), gas-clock (simulation step), gas-scan (scan cycle)
transmission/   gas-trunk (high pressure line)
stations/
  station-i/    gas-sensor-i, gas-valve-i, gas-ctrl-i, gas-plant-i
consumers/      gas-consumer-i + gas-demand-gen-i (demand profile)
safety/         gas-leak-monitor
supervisory/    gas-network (profile, network setpoint, load), gas-stats
rules/
```

```
  control/      gas-rule-pi-i, gas-rule-mode-guard-i
  safety/       gas-rule-esd-trip-i, gas-rule-esd-reset-i, gas-rule-leak-detect
  process/      gas-rule-plant-i
  supervisory/  gas-rule-balancer, gas-rule-stats, gas-rule-bad-sensor
compute/        gas-control-loop, gas-process, gas-supervisory
links/          link-to and link-from edges
triggers/       fire edges
```

Vertex	Main properties	Role
`gas-plant-i`	`pressure`, `upstream`	simulated process: the real downstream pressure
`gas-sensor-i`	`pressure`	transmitter: sees the pressure with dead time and deadband
`gas-ctrl-i`	`setpoint`, `mode`, `fault`, `integral`, `output`, `bias`, `kp`, `ki`, `esd-trips`	loop controller state
`gas-valve-i`	`opening` (0-100 %)	control valve
`gas-consumer-i`	`demand`	demand of the station

DATA FLOW OF A STATION

```
gas-gate-gen --link-to--> gas-trunk --link-to--> gas-plant-i.upstream
gas-demand-gen-i --link-to--> gas-consumer-i.demand
                  rule "plant" (valve opening, upstream, demand) -> gas-plant-i.pressure
gas-plant-i.pressure --link-to (delay + deadband)--> gas-sensor-i.pressure --fire--> PI / guard / ESD rules
gas-network.setpoint --link-to--> gas-ctrl-i.setpoint
gas-ctrl-i.output --link-from (deadband 0.5 %)--> gas-valve-i.opening --> rule "plant"
```

The loop is closed: the valve changes the pressure, the transmitter sees it later, the controller corrects the valve.

Rules

Rule	Fired by	Condition	Action
`gas-rule-pi-i`	`sensor.pressure` and scan cycle	mode `AUTO`	error, integral with anti-windup, `output` 0-100 %
`gas-rule-mode-guard-i`	`sensor.pressure`	fault state changes	measure < 0 or > 200: `fault` = `sensor`, mode `MANUAL`; back to `AUTO` with a valid measure
`gas-rule-esd-trip-i`	`sensor.pressure`	pressure > setpoint × (1 + ESD margin)	mode `ESD`, output 0, `esd-trips` +1
`gas-rule-esd-reset-i`	`sensor.pressure`	mode `ESD` and pressure < 50 % of the setpoint	back to `AUTO`
`gas-rule-plant-i`	clock	always	process model
`gas-rule-balancer`	clock	always	network setpoint from the mean demand (hysteresis 1 s)
`gas-rule-stats`	clock	always	mean error, mean opening, total ESD trips (hysteresis 1 s)
`gas-rule-leak-detect`	mean opening	mean opening > 90 % changes the alarm state	`alarm` = `leak` / `none`
`gas-rule-bad-sensor`	clock	reads a missing transmitter	fails on purpose: `errors` grows

PI controller: `error = setpoint - pressure`, `output = clamp(bias + kp·error + ki·integral, 0, 100)` with `kp` = 0.4, `ki` = 0.1; the integral is not updated when the output is saturated.

Process model, at every clock step: `target = 0.9 · opening · (upstream / 24) · flow_factor - 0.25 · (demand - nominal) - 0.25 · leak / stations`, `pressure = pressure + 0.3 · (target - pressure)`.

Compute	Threads	Rules (priority)	Why
`gas-control-loop`	1	ESD trip (300), guard (250), ESD reset (200), PI (100)	controllers must not overlap, safety first
`gas-process`	4	plant models (100)	independent, in parallel
`gas-supervisory`	1	leak (60), balancer (50), stats (40), bad-sensor (10)	slow logic, low priority

Safety rules have no hysteresis: a discarded notification could hide an overpressure.

What to observe

Values after about one minute with the defaults:

Station	`sensor.pressure`	`ctrl.setpoint`	`ctrl.mode`	`ctrl.output`	`valve.opening`	PI `tota`
0 (deadband)	23.76	25.2	AUTO	29.1	28.95	138
3 (no deadband)	24.51	25.2	AUTO	30.19	30.36	398

- The pressure follows the setpoint; `valve.opening` follows `ctrl.output` through the `link-from` edge.
- Station 3 has no deadband: its PI rule is evaluated about three times more often for the same regulation.
- `gas-stats`: `mean-abs-error` 2.75, `mean-opening` 29.44, `esd-trips` 0.
- `gas-rule-bad-sensor`: `errors` = `total`, `executed` = 0 (JEXL exceptions in the log are expected for this rule).

Live interventions

Verified on the default scenario:

Intervention	Result
write 90 on `gas-sensor-1.pressure`	immediately `gas-ctrl-1.mode` = `ESD`, `output` = 0, `esd-trips` = 1; the other stations stay `AUTO`; after a few seconds the pressure falls and the controller is back in `AUTO`
write -5 on `gas-sensor-1.pressure`	`fault` = `sensor`, mode `MANUAL` until the next valid measure from the plant (a few hundred ms)
write 600 on `gas-leak-monitor.leak`	the valves open to compensate; when `gas-stats.mean-opening` exceeds 90 % (about 40 s) `gas-leak-monitor.alarm` = `leak`; write 0 to cancel
write 40 on `gas-network.base-setpoint`	the balancer distributes the new setpoint (41.2 with the current demand) to all controllers; the pressures follow (43.8 after 25 s)
change `kp`, `ki` of `gas-ctrl-i`	faster or more oscillating response
change `deadband-value` or `delay` of the edge `pipe-to-transmitter-i`	more or fewer PI executions, more or less oscillation
write `MANUAL` on `gas-ctrl-i.mode`	the PI rule stops acting
`gas-demand-gen-i.algorithm` = `stopped`	the demand stays constant

Rules engine concepts in the demo

Concept	Where
`condition` / `action`	PI, guard, ESD, leak
`fire` edge	`triggers/` : the measure fires the controllers
`rules:compute` with `threads`	1 thread for the loops, 4 for the process
`rules:execute` with `priority`	ESD (300) before PI (100)
hysteresis	balancer and statistics (1 s)
counters	`queue` , `total` , `executed` , `errors` on every rule
runtime changes	`setpoint` , `kp` , `ki` , `deadband-value` , `delay`

Limits

- No pressure losses, line pack, temperature or real gas composition.
- Upstream pressure is fixed around 24 bar with a slow oscillation.
- The PI is discrete with a variable step (executions depend on events) and is not tuned for a real plant.
- `metano-blu` changes only some coefficients (flow, demand, thresholds, delay).

To create the scenario again restart the server or remove the `gas-...` vertices (see Operations). Tests: `WaldotRulesEngineGasGridTest` .

4. Developers

4.1 WaldOT Developer Guide

Hands-on development guide for software engineers

4.1.1 Quick Start

Prerequisites

- JDK 26 (the build targets Java 26; virtual threads are used throughout)
- Docker (for containerized deployment)
- Git
- IDE with Java support (IntelliJ IDEA, Eclipse, VS Code)

5-Minute Setup

```
# Clone repository
git clone https://github.com/rossonet/waldot.git
cd waldot

# Build project
./gradlew clean build

# Run WaldOT
java -jar waldot-app/build/libs/waldot-app-*.jar

# Access OPC UA server
# opc.tcp://localhost:12686/waldot
```

Docker Quick Start

```
# Pull and run
docker pull rossonet/waldot:latest
docker run -p 12686:12686 -p 8443:8443 rossonet/waldot:latest

# Access
# OPC UA: opc.tcp://localhost:12686/waldot
# HTTPS: https://localhost:8443
```

4.1.2 Core Concepts

1. WaldotGraph

The main interface combining TinkerPop Graph with OPC UA integration.

```
// Open graph
WaldotGraph graph = OpcFactory.getOpcGraph(
    "file:///tmp/waldot.db",
    new LoggerHistoryStrategy()
);

// Add vertex
Vertex v = graph.addVertex();
v.property("label", "sensor1");
v.property("temperature", 25.5);

// Query
List<Vertex> sensors = graph.traversal()
    .V()
    .has("label", "sensor1")
    .toList();

// Close
graph.close();
```

2. WaldotVertex

Represents both a TinkerPop Vertex and an OPC UA Object.

```
WaldotVertex sensor = (WaldotVertex) graph.addVertex();
sensor.property("label", "temp-sensor");
sensor.property("value", 20.0);

// Add edge
WaldotVertex zone = (WaldotVertex) graph.addVertex();
zone.property("label", "production-zone");
sensor.addEdge("located-in", zone);

// Observe property changes
sensor.addPropertyObserver((vertex, key, oldValue, newValue) -> {
    System.out.println(key + " changed: " + oldValue + " → " + newValue);
});
```

3. Gremlin Queries

```
GraphTraversalSource g = graph.traversal();

// Find all sensors
List<Vertex> sensors = g.V()
    .has("type", "sensor")
    .toList();

// Find sensors in alarm
List<String> alarms = g.V()
    .has("type", "sensor")
    .has("status", "alarm")
    .values("label")
    .toList();

// Complex traversal
List<Path> paths = g.V()
    .has("label", "Line1")
    .out("contains")
    .has("type", "motor")
    .has("status", "alarm")
    .path()
    .by("label")
    .toList();
```

4.1.3 Plugin Development

Plugins are discovered at startup: a top-level class annotated with `@WaldotPlugin`, with a public no-argument constructor, located in a package under `net.rossonet.waldot`, `plugin` or `plugins`, usually extending `AbstractPluginListener`. It registers vertex types (`registerTypeNode(...)`), edge types (`registerTypeEdge(...)`) and commands (`registerCommand(...)`) in `initialize(WaldotNamespace)`.

The complete, verified walkthrough (project setup, vertex hooks, custom edges, commands, packaging in the Docker image, tests) is in the **Integration Guide**.

4.1.4 Working with the base plugins

Vertices of the base plugins are created with `graph.addVertex` in a bootstrap script, in the `query` command or in Java code. Keys are lowercase; scripts are JEXL and statements end with `;`.

Generator

```
// sinusoidal temperature, new value in 'data' every 5 s
var temp = graph.addVertex('id', 'office-temp', 'type', 'sim:generator', 'label', 'office-temp',
    'algorithm', 'sinusoidal', 'min', '18', 'max', '26', 'delay', '5000');
// production counter, +1 every minute
var counter = graph.addVertex('id', 'items-produced', 'type', 'sim:generator', 'label', 'items-produced',
    'algorithm', 'incremental', 'min', '0', 'max', '1000000', 'delay', '60000');
log.info('temperature: ' + temp.value('data'));
```

Rules engine

```
var compute = graph.addVertex('id', 'main-compute', 'type', 'rules:compute', 'label', 'main-compute', 'threads', '4');
var rule = graph.addVertex('id', 'temp-alarm', 'type', 'rules:rule', 'label', 'temp-alarm',
    'condition', "g.V().has('id','office-temp').values('data').next() > 25.0",
    'action', "log.warn('High temperature')",
    'hysteresis', '5000');
compute.addEdge('runner', rule, 'type', 'rules:execute', 'priority', '100');
temp.addEdge('watch', rule, 'type', 'fire', 'properties-to-monitor', 'data');
```

The same code in Java:

```
Vertex compute = graph.addVertex("id", "main-compute", "type", "rules:compute", "label", "main-compute", "threads", "4");
Vertex rule = graph.addVertex("id", "temp-alarm", "type", "rules:rule", "label", "temp-alarm",
        "condition", "g.V().has('id','office-temp').values('data').next() > 25.0",
        "action", "log.warn('High temperature')", "hysteresis", "5000");
compute.addEdge("runner", rule, "type", "rules:execute", "priority", "100");
temp.addEdge("watch", rule, "type", "fire", "properties-to-monitor", "data");
```

Variables available in `condition` and `action`:

Variable	Description
`log`	logger
`g`	Gremlin traversal source
`graph`	the WaldOT graph
`cmd`	console helper (`getVerticesCount()` , `toNumber(x)` , ...)
`self`	the rule vertex
`math`	`java.lang.Math`
`rand`	`java.util.Random`
commands	every console command, as `<folder>_<name>`

Properties, weights, counters and debug: rules engine README and User Guide.

Bootstrap

Syntax, format detection and troubleshooting: Bootstrap Configuration. Complete verified scripts: Examples.

4.1.5 Testing

Unit Testing Plugins

```
@Test
public void testCustomVertex() {
    // Create test graph
    WaldotGraph graph = OpcFactory.getOpcGraph(
        "file:///tmp/test.db",
        new LoggerHistoryStrategy()
    );

    // Create vertex
    Vertex v = graph.addVertex("type", "my:custom");
    v.property("value", 42.0);

    // Assert
    assertEquals(42.0, v.property("value").value());

    // Cleanup
    graph.close();
}
```

Integration Testing with Testcontainers

```
@Test
public void testWaldotContainer() {
    try (GenericContainer<?> waldot = new GenericContainer<>("rossonet/waldot:latest")
```

```
            .withExposedPorts(12686)) {

        waldot.start();

        // Connect OPC UA client
        String endpoint = "opc.tcp://" + waldot.getHost() + ":" +
                          waldot.getMappedPort(12686) + "/waldot";

        // Test connection
        // ...
    }
}
```

4.1.6 Best Practices

1. Use Virtual Threads for I/O

```
// ✅ GOOD: Virtual thread for blocking I/O
ExecutorService executor = ThreadHelper.newVirtualThreadExecutor();
executor.submit(() -> {
    while (active) {
        String data = httpClient.get(url).block();  // Blocking OK
        processData(data);
        Thread.sleep(1000);  // Blocking OK
    }
});

// ❌ BAD: Platform thread pool
ExecutorService executor = Executors.newFixedThreadPool(10);  // Limited scalability
```

2. Handle Interruptions

```
// ✅ GOOD: Proper interruption handling
while (active) {
    try {
        doWork();
        Thread.sleep(1000);
    } catch (InterruptedException e) {
        active = false;  // Exit gracefully
        Thread.currentThread().interrupt();  // Restore interrupt status
    }
}

// ❌ BAD: Ignoring interruptions
while (true) {
    doWork();
    Thread.sleep(1000);  // Exception ignored
}
```

3. Clean Up Resources

```
// ✅ GOOD: Proper cleanup
@Override
public void close() throws Exception {
    active = false;
    executor.shutdownNow();
    if (!executor.awaitTermination(5, TimeUnit.SECONDS)) {
        logger.warn("Executor did not terminate");
    }
}
```

4. Use Property Observers

```
// ✅ GOOD: React to property changes
vertex.addPropertyObserver((v, key, oldValue, newValue) -> {
    if ("temperature".equals(key) && (Double)newValue > 80.0) {
        logger.warn("High temperature detected");
    }
});
```

4.1.7 Debugging

Enable Debug Logging

```
# Docker
docker run \
  -e JDK_JAVA_OPTIONS="-Dorg.slf4j.simpleLogger.defaultLogLevel=debug" \
  rossonet/waldot:latest

# Java (fat jar built with ./gradlew :waldot-app:shadowJar)
java -Dorg.slf4j.simpleLogger.defaultLogLevel=debug \
     -jar waldot-app/build/libs/waldot-app-<version>-all.jar
```

Debug Rules

```
// enable debug: 0 = off, 1 = OPC UA events, 2 = events and log
g.V().has('id', 'temp-alarm').next().property('debug', 2);

// counters
var rule = g.V().has('id', 'temp-alarm').next();
log.info('queue=' + rule.value('queue') + ' total=' + rule.value('total')
    + ' executed=' + rule.value('executed') + ' errors=' + rule.value('errors'));
```

Connect with OPC UA Client

- **UaExpert**: https://www.unified-automation.com/products/development-tools/uaexpert.html
- **Prosys Browser**: https://www.prosysopc.com/products/opc-ua-browser/

4.1.8 API Reference

Key Interfaces

- `WaldotGraph` - Main graph interface
- `WaldotVertex` - Vertex interface
- `WaldotEdge` - Edge interface
- `WaldotNamespace` - Namespace management
- `PluginListener` - Plugin extension point

Key Classes

- `AbstractOpcVertex` - Base class for custom vertices
- `AbstractOpcEdge` - Base class for custom edges
- `ThreadHelper` - Virtual thread utilities
- `OpcFactory` - Graph factory

Complete Documentation

- Integration Guide - How to write a plugin
- API JavaDoc

4.1.9 Examples

- Docker Compose examples - server deployment variants
- Rules engine showcase: gas grid
- Generator quick start
- Configuration Reference

4.1.10 Community

- **GitHub**: https://github.com/rossonet/waldot
- **Issues**: https://github.com/rossonet/waldot/issues
- **Discussions**: https://github.com/rossonet/waldot/discussions

---

Happy coding! 🚀

4.2 WaldOT Integration Guide: Writing a Plugin

This guide explains how to extend WaldOT with a **plugin**: a Java library that adds new *active* vertex types, edge types and commands to the graph. Every example on this page has been compiled against the libraries of the `rossonet/waldot` image and executed in a container.

The built-in plugins of the `waldot-app` distribution are the reference implementations:

Plugin	Module	Vertex types	Edge types	Commands (folder)
Generator	`plugins/waldot-plugin-generator`	`sim:generator`, `sim:event-generator`	—	`create generator`, `create event generator` (`simulator`)
Rules engine	`plugins/waldot-plugin-rules-engine`	`rules:rule`, `rules:compute`	`rules:execute`	`create rule`, `create compute`, `create compute monitored edge`, showcases (`rules`)
TinkerPop	`plugins/waldot-plugin-tinkerpop`	`gremlin`	—	`create gremlin server` (`tinkerpop`)

4.2.1 1. What a plugin can do

In WaldOT every vertex of the graph is also an OPC UA object and every vertex property is an OPC UA variable. A plugin contributes:

- **vertex types**: a Java class (subclass of `AbstractOpcVertex`) plus an OPC UA `ObjectType`. When a vertex is created with `type = <your label>` WaldOT asks your plugin to build it. The vertex can run code: react to property writes, receive notifications through `fire` edges, run background threads, publish OPC UA events;
- **edge types**: labels for which your plugin is notified when an edge is added or removed, with the possibility to veto the creation;
- **commands**: OPC UA methods under the `Commands` folder that are also callable from bootstrap scripts, rule actions and the query console.

Everything a plugin does is visible from any OPC UA client: the vertex variables, the events it posts, the commands it exposes.

4.2.2 2. Discovery and lifecycle

Discovery

At startup `WaldotOpcUaServer` scans the class path for **top-level classes** annotated with `@WaldotPlugin` in these packages (and their sub-packages):

```
net.rossonet.waldot    plugin    plugins
```

The class must implement `PluginListener` (in practice: extend `AbstractPluginListener`) and have a **public no-argument constructor**.

> **Warning**
>
> A class in any other package (for example `com.mycompany.waldot`) is **not** discovered. Use `plugins.<vendor>.<name>` for third-party plugins.

The log confirms the discovery:

```
INFO net.rossonet.waldot.opc.WaldotOpcUaServer - Found plugin: plugins.example.ThresholdPlugin
INFO net.rossonet.waldot.namespaces.HomunculusNamespace - Registering commands from plugin ThresholdPlugin
```

Lifecycle

Phase	Method	What to do
Registration	`initialize(WaldotNamespace)`	Register vertex types, edge types and commands. Called once, before the OPC UA server starts.
Command registration	`getCommands()`	Returns the commands to publish (filled by `registerCommand(...)`). Read right after `initialize`.
Start	`start()`	Called just before the bootstrap file is executed (and again after a namespace reset).
Vertex creation	`containsVertexType(label)`, `getVertexTypeNode(label)`, `containsVertexTypeNode(nodeId)`, `createVertex(...)`	Handled by `AbstractPluginListener` once the type is registered.
Edge creation / removal	`containsEdgeType(label)`, `notifyAddEdge(...)`, `notifyRemoveEdge(...)`	Called only for the edge types you registered. Throw `PluginBlockingException` to refuse an edge.
Reset	`reset()`	Namespace reset: clear plugin state; the bootstrap runs again afterwards.
Shutdown	`stop()`	Stop background activity of the plugin.

4.2.3 3. Project setup

All WaldOT modules target **Java 26**. A plugin only needs `waldot-api` at compile time: it brings Eclipse Milo, Apache TinkerPop and the WaldOT base classes.

Option A: module inside the WaldOT repository

1. Create `plugins/waldot-plugin-<name>/` with `src/main/java/...`.
2. Register it in `settings.gradle`:

```
include 'waldot-plugin-<name>'
project(':waldot-plugin-<name>').projectDir = file('plugins/waldot-plugin-<name>')
```

3. Minimal `build.gradle` (same layout as the built-in plugins):

```
group = 'net.rossonet.waldot'
version = "${rootProject.version}"

apply plugin: 'java-library'

java {
    sourceCompatibility = JavaVersion.VERSION_26
    targetCompatibility = JavaVersion.VERSION_26
}

repositories {
    mavenCentral()
    maven { url 'https://jitpack.io' }
}

dependencies {
```

```
    implementation project(':waldot-deps')
    api project(':waldot-api')
    testImplementation project(':waldot-namespace')
    testImplementation testFixtures(project(':waldot-client'))
    testImplementation(platform('org.junit:junit-bom:5.13.1'))
    testImplementation('org.junit.jupiter:junit-jupiter')
    testRuntimeOnly('org.junit.platform:junit-platform-launcher')
}
```

4. Add it to the application in `waldot-app/build.gradle`:

```
implementation project(':waldot-plugin-<name>')
```

The next `docker buildx build` produces an image that contains it.

Option B: external jar added to an existing image

The image starts WaldOT with the class path `/app/lib/*`: any jar placed in `/app/lib` is loaded and scanned for plugins.

1. Compile against the WaldOT libraries (for example the jars of the image):

```
mkdir -p lib
docker run --rm --entrypoint sh -v "$PWD/lib:/out" rossonet/waldot:latest \
  -c 'cp /app/lib/*.jar /out/'
javac --release 26 -cp "lib/*" -d out $(find src -name '*.java')
jar --create --file my-plugin.jar -C out .
```

2. Mount the jar (development):

```
docker run -p 12686:12686 \
  -v "$PWD/my-plugin.jar:/app/lib/my-plugin.jar:ro" \
  -v "$PWD/boot.conf:/waldot/boot.conf:ro" \
  rossonet/waldot:latest
```

or build a derived image (production):

```
FROM rossonet/waldot:latest
COPY my-plugin.jar /app/lib/
```

Third-party libraries not already present in `/app/lib` must be shipped with the plugin (extra jars in `/app/lib` or a shaded jar).

4.2.4 4. Step by step: a threshold plugin

The example adds the vertex type `example:threshold`. The vertex compares its `value` with its `limit`, keeps `alarm` and `exceeded-count` up to date and posts an OPC UA event on every transition. `value` can be written by an OPC UA client, by a rule, by a Gremlin query or, more interestingly, by a `link-to` edge coming from another vertex.

```
src/main/java/plugins/example/
├── ThresholdPlugin.java         plugin: registers type and command
├── ThresholdVertex.java         active vertex
└── CreateThresholdCommand.java  OPC UA / console command
```

4.1 The plugin class

```
package plugins.example;

import net.rossonet.waldot.api.annotation.WaldotPlugin;
import net.rossonet.waldot.api.models.WaldotNamespace;
import net.rossonet.waldot.plugin.AbstractPluginListener;

@WaldotPlugin
public class ThresholdPlugin extends AbstractPluginListener {

    /** Value of the "type" property that selects this plugin. */
    public static final String TYPE_LABEL = "example:threshold";

    /** Folder of the plugin commands under the OPC UA "Commands" node. */
    public static final String COMMAND_DIRECTORY = "example";

    @Override
    public void initialize(final WaldotNamespace waldotNamespace) {
        super.initialize(waldotNamespace);
```

```
        commands.clear();
        // OPC UA ObjectType + Java class + type label
        registerTypeNode(ThresholdVertex.class, "ExampleThresholdObjectType",
                "Example Threshold", TYPE_LABEL);
        waldotNamespace.getOpcuaServer().updateReferenceTypeTree();
        // OPC UA method Commands/example/"create threshold",
        // console function example_create_threshold
        registerCommand(new CreateThresholdCommand(waldotNamespace));
        logger.info("Example threshold plugin initialized");
    }
}
```

`registerTypeNode(vertexClass, typeNodeId, displayName, typeLabel)`:

- creates the OPC UA `ObjectType` `ObjectTypes/<typeNodeId>` (subtype of `BaseObjectType`) with a `Label` variable;
- calls the **static** method `generateParameters(WaldotNamespace, UaObjectTypeNode)` of the vertex class (by reflection) to describe the other variables of the type;
- binds `typeLabel` to the class: `AbstractPluginListener.createVertex(...)` then instantiates the class through the standard constructor shown below.

If the vertex needs extra constructor arguments (for example a shared executor), override `createVertex(...)` as `WaldotGeneratorPlugin` does for `sim:generator`.

4.2 The vertex class

```
package plugins.example;

import org.eclipse.milo.opcua.sdk.server.model.objects.BaseEventType;
import org.eclipse.milo.opcua.sdk.server.nodes.UaNode;
import org.eclipse.milo.opcua.sdk.server.nodes.UaNodeContext;
import org.eclipse.milo.opcua.sdk.server.nodes.UaObjectTypeNode;
import org.eclipse.milo.opcua.stack.core.NodeIds;
import org.eclipse.milo.opcua.stack.core.types.builtin.DataValue;
import org.eclipse.milo.opcua.stack.core.types.builtin.LocalizedText;
import org.eclipse.milo.opcua.stack.core.types.builtin.NodeId;
import org.eclipse.milo.opcua.stack.core.types.builtin.QualifiedName;
import org.eclipse.milo.opcua.stack.core.types.builtin.unsigned.UByte;
import org.eclipse.milo.opcua.stack.core.types.builtin.unsigned.UInteger;

import net.rossonet.waldot.api.PluginListener;
import net.rossonet.waldot.api.models.WaldotGraph;
import net.rossonet.waldot.api.models.WaldotNamespace;
import net.rossonet.waldot.opc.AbstractOpcVertex;

public class ThresholdVertex extends AbstractOpcVertex implements AutoCloseable {

    public static final String ALARM = "alarm";
    public static final String COUNT = "exceeded-count";
    public static final String LIMIT = "limit";
    public static final String VALUE = "value";

    /** Called by registerTypeNode(...): variables of the OPC UA ObjectType. */
    public static void generateParameters(final WaldotNamespace namespace,
            final UaObjectTypeNode typeNode) {
        PluginListener.addParameterToTypeNode(namespace, typeNode, LIMIT, NodeIds.Double, "alarm threshold");
        PluginListener.addParameterToTypeNode(namespace, typeNode, VALUE, NodeIds.Double, "monitored value");
        PluginListener.addParameterToTypeNode(namespace, typeNode, ALARM, NodeIds.Boolean, "true when value > limit");
        PluginListener.addParameterToTypeNode(namespace, typeNode, COUNT, NodeIds.Int64,
                "number of times the limit was exceeded");
    }

    private boolean inAlarm = false;

    /** Constructor signature required by AbstractPluginListener.createVertex(...). */
    public ThresholdVertex(final WaldotGraph graph, final UaNodeContext context, final NodeId nodeId,
            final QualifiedName browseName, final LocalizedText displayName, final LocalizedText description,
            final UInteger writeMask, final UInteger userWriteMask, final UByte eventNotifier, final long version,
            final Object[] propertyKeyValues) {
        super(graph, context, nodeId, browseName, displayName, description, writeMask, userWriteMask,
                eventNotifier, version);
        // managed properties: typed, with default value and optional validation
        addManagedProperty(LIMIT, NodeIds.Double, Double.class, false, 100.0, (label, oldValue, newValue) -> {
            if (newValue == null || newValue.isNaN()) {
                throw new PropertyChangeException("limit must be a number");
            }
        }, false, propertyKeyValues);
        addManagedProperty(VALUE, NodeIds.Double, Double.class, false, 0.0, null, false, propertyKeyValues);
        addManagedProperty(ALARM, NodeIds.Boolean, Boolean.class, false, false, null, false, propertyKeyValues);
        addManagedProperty(COUNT, NodeIds.Int64, Long.class, false, 0L, null, false, propertyKeyValues);
    }

    private synchronized void check() {
        final Double value = getRuntimeParameter(VALUE);
        final Double limit = getRuntimeParameter(LIMIT);
        if (value == null || limit == null) {
```

```
            return;
        }
        final boolean alarm = value > limit;
        if (alarm != inAlarm) {
            inAlarm = alarm;
            updatePropertyIfChanged(ALARM, alarm);
            if (alarm) {
                final Long count = getRuntimeParameter(COUNT);
                updatePropertyIfChanged(COUNT, (count == null ? 0L : count) + 1);
            }
            // OPC UA event, visible to every client subscribed to the vertex
            sendLogEvent("threshold", (alarm ? "limit exceeded: " : "back to normal: ") + value + " / " + limit,
                    alarm ? 700 : 100);
        }
    }

    /** Required by GremlinElement. */
    @Override
    public Object clone() {
        return new ThresholdVertex(graph, getNodeContext(), getNodeId(), getBrowseName(), getDisplayName(),
                getDescription(), getWriteMask(), getUserWriteMask(), getEventNotifier(), version(),
                getPropertiesAsStringArray());
    }

    @Override
    public void close() {
        // release threads, sockets, files... (nothing to do here)
    }

    /** A "fire" edge pointing to this vertex delivers source events here. */
    @Override
    public void fireEvent(final UaNode node, final BaseEventType event, final int priority) {
        check();
    }

    /** A "fire" edge pointing to this vertex delivers source property changes here. */
    @Override
    public void fireProperty(final UaNode node, final String propertyLabel, final DataValue value,
            final int priority) {
        check();
    }

    /** Every property write (OPC UA client, Gremlin, rule, link edge) passes here. */
    @Override
    public void notifyPropertyValueChanging(final String label, final DataValue value) {
        super.notifyPropertyValueChanging(label, value); // keeps managed properties in sync
        if (VALUE.equals(label) || LIMIT.equals(label)) {
            check();
        }
    }

    @Override
    public void notifyRemoveVertex() {
        close();
    }
}
```

4.3 The creation command

```
package plugins.example;

import java.util.List;

import net.rossonet.waldot.api.models.WaldotNamespace;
import net.rossonet.waldot.opc.AbstractCreateVertexCommand;

public class CreateThresholdCommand extends AbstractCreateVertexCommand {

    public CreateThresholdCommand(final WaldotNamespace waldotNamespace) {
        super(waldotNamespace, "create threshold", "Create an example threshold vertex",
                ThresholdPlugin.COMMAND_DIRECTORY, ThresholdPlugin.TYPE_LABEL, Language.ENGLISH,
                List.of(VertexParameter.optional(ThresholdVertex.LIMIT, VariableNodeTypes.Double,
                        "alarm threshold, default 100")));
    }

    @Override
    protected String validate(final List<Object> keyValues) {
        final String limit = findValue(keyValues, ThresholdVertex.LIMIT);
        if (limit != null && Double.parseDouble(limit) < 0) {
            return "the limit must not be negative";
        }
        return null;
    }
}
```

4.4 Using the new vertex

Bootstrap file (script mode, JEXL syntax):

```
// a generator feeds the threshold vertex through a link-to edge
gen = graph.addVertex('id', 'gen-1', 'type', 'sim:generator', 'label', 'gen-1', 'directory', 'example',
    'algorithm', 'sinusoidal', 'min', '0', 'max', '200', 'delay', '300');
thr = graph.addVertex('id', 'thr-1', 'type', 'example:threshold', 'label', 'thr-1', 'directory', 'example',
    'limit', '150');
gen.addEdge('feeds', thr, 'type', 'link-to', 'properties-to-monitor', 'data', 'data', 'value');

// the same vertex type created through the plugin command
log.info('created by command: ' + example_create_threshold.exec('thr-2', 'example', '50')[0]);
log.info('invalid: ' + example_create_threshold.exec('thr-3', 'example', '-1')[0]);
```

Observed result:

```
INFO AbstractOpcCommand - create vertex with parameters: [type, example:threshold, label, thr-2, directory, example, limit, 50]
INFO CONSOLE - invalid: error: the limit must not be negative
```

Reading `ns=2;s=thr-1` with an OPC UA client every two seconds:

```
{'limit': 150.0, 'value': 66.17,  'alarm': False, 'exceeded-count': 20, 'type': 'example:threshold'}
{'limit': 150.0, 'value': 136.32, 'alarm': False, 'exceeded-count': 20, 'type': 'example:threshold'}
{'limit': 150.0, 'value': 188.59, 'alarm': True,  'exceeded-count': 22, 'type': 'example:threshold'}
```

and the folder `Commands/example` contains the method `create threshold`.

4.2.5 5. Writing vertices

Constructor and identity

- The constructor must have exactly the signature shown above (`WaldotGraph, UaNodeContext, NodeId, QualifiedName, LocalizedText, LocalizedText, UInteger, UInteger, UByte, long, Object[]`) when you rely on `registerTypeNode(...)`.
- `clone()` is abstract in `GremlinElement`: implement it with the same constructor.
- `propertyKeyValues` contains the key/value pairs passed to `graph.addVertex(...)` or to the command.
- Reserved keys handled by WaldOT: `id` (OPC UA identifier, for example `ns=2;s=thr-1`), `type`, `label`, `directory` (OPC UA folder, levels separated by `/`), `name`, `description`, history flags. `label` and `type` cannot be changed after creation.
- Use **lowercase** property keys: keys arrive lowercase in `notifyPropertyValueChanging` and the built-in plugins register their managed properties in lowercase.

Managed properties

```
addManagedProperty(label, opcUaDataType, javaType, isRequired, defaultValue,
       callbackOnChange, isNecessaryRestartVertex, propertyKeyValues);
```

- The initial value is taken from `propertyKeyValues` and converted to `javaType` (`String`, `Integer`, `Long`, `Boolean`, `Double`, `Float`, `Byte`, `Instant`, `JSONObject`); if absent or not convertible the default is used.
- Every later write (OPC UA client, Gremlin `property(...)`, rule, link edge) goes through the conversion and, if present, through `callbackOnChange`; throwing `PropertyChangeException` rejects the value.
- `getRuntimeParameter(label)` returns the current typed value; `setRuntimeParameter(label, value)` updates it without notifications.
- `property(key, value)` / `updatePropertyIfChanged(key, value)` write a graph property: the OPC UA variable is updated and observers (`link-*` and `fire` edges, history) are notified.

Reacting to the graph

Hook	Triggered by
`notifyPropertyValueChanging(label, value)`	Any write of a property of this vertex. **Always call** `super` first: it keeps label/type immutable, updates managed properties, notifies observers and history.
`fireProperty(node, label, value, priority)`	A `fire` edge whose source changed one of its `properties-to-monitor`.
`fireEvent(node, event, priority)`	A `fire` edge whose source posted an OPC UA event.
`notifyRemoveVertex()`	The vertex is being removed: stop threads and release resources.

`link-to` and `link-from` edges write directly into the target (or source) properties, so a vertex fed by a link edge sees the change in `notifyPropertyValueChanging`. These hooks run on the caller thread: keep them short and move long or blocking work to a thread of the vertex.

Background work

Active vertices that poll a device or produce data run their own loop on a **virtual thread**, started in the constructor and stopped in `close()` / `notifyRemoveVertex()`:

```
runner = Thread.ofVirtual().name("my-vertex-" + getNodeId().toParseableString()).unstarted(this::loop);
runner.start();
```

Read the parameters at every cycle (as `sim:event-generator` does) so that OPC UA clients and rules can change them at runtime.

Publishing OPC UA events

- `sendLogEvent(category, message, severity)` posts a `SystemEventType` event whose source is the vertex (severity 1–1000).
- `postEvent(BaseEventTypeNode)` posts an event built with `getNamespace().getEventFactory()`.

Events reach subscribed OPC UA clients and every `fire` edge leaving the vertex, so they can trigger rules.

4.2.6 6. Custom edge types

Built-in edge behaviors (`link-to`, `link-from`, `fire`) are provided by the namespace. A plugin declares its own edge labels with `registerTypeEdge("<prefix>:<name>")` in `initialize(...)`. For those labels:

- `notifyAddEdge(edge, source, target, label, type, keyValues)` is called after the edge is created; throwing `PluginBlockingException` removes it and the caller receives an error;
- `notifyRemoveEdge(edge)` is called on removal;
- `AbstractPluginListener` forwards both notifications to source and target vertices that implement `EdgeMonitoredVertex` (`notifyAddTargetMonitoredEdge`, `notifyAddSourceMonitoredEdge`, ...).

The rules engine uses this mechanism for `rules:execute`: `ComputeVertex` implements `EdgeMonitoredVertex` and refuses the edge when the target is not a rule:

```
@Override
public void notifyAddTargetMonitoredEdge(WaldotEdge edge, WaldotVertex targetVertex,
        String label, String type, Object[] propertyKeyValues) throws PluginBlockingException {
    if (!RulesEngineConstants.EXECUTE_EDGE_LABEL.equals(type)) {
        return;
    }
    if (!(targetVertex instanceof ComputableFireableAbstractOpcVertex)) {
        throw new PluginBlockingException("The target of a " + type
                + " edge must be a rule vertex, but it is " + targetVertex);
    }
    // ... start serving the rule
}
```

4.2.7 7. Commands

Vertex creation commands

`AbstractCreateVertexCommand` builds a complete OPC UA method from a list of `VertexParameter` s:

- inputs: `label` , `directory` , then one argument per parameter (`VertexParameter.optional(...)` / `VertexParameter.required(...)` with an OPC UA type from `VariableNodeTypes`);
- output: `result` , the NodeId of the new vertex or a message starting with `error:` ;
- empty inputs are skipped (the vertex uses its defaults), numeric and boolean values are checked, then `validate(keyValues)` runs your own checks;
- `clone()` uses the constructor with the `WaldotNamespace` as only argument.

Where commands appear

Access	Name
OPC UA	method `Commands/<directory>/<command name>` , e.g. `Commands/example/create threshold`
Bootstrap scripts, rule actions, `query` console	object `<directory>_<command name>` with spaces replaced by `_` , invoked with `.exec(...)` : `example_create_threshold.exec('thr-2', 'example', '50')`

For commands that are not vertex factories extend `AbstractOpcCommand` and implement `runCommand(InvocationContext, String[])` .

4.2.8 8. Testing

Plugin tests start a complete in-process namespace with `OpcFactory` (module `waldot-namespace`) and the test fixtures of `waldot-client` ; the plugin is discovered from the test class path like in production:

```
package plugins.example;

import static org.junit.jupiter.api.Assertions.assertEquals;

import java.nio.file.Files;

import org.apache.tinkerpop.gremlin.structure.Vertex;
import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import net.rossonet.waldot.api.models.WaldotGraph;
import net.rossonet.waldot.gremlin.opcgraph.strategies.opcua.history.LoggerHistoryStrategy;
import net.rossonet.waldot.gremlin.opcgraph.structure.OpcFactory;
import net.rossonet.waldot.test.WaldotTestEnvironment;

public class ThresholdVertexTest {

    private WaldotGraph g;

    @BeforeEach
    public void beforeEach() throws Exception {
        Files.deleteIfExists(WaldotTestEnvironment.bootFile());
        g = OpcFactory.getOpcGraph(WaldotTestEnvironment.bootUrl(), new LoggerHistoryStrategy());
    }

    @AfterEach
    public void afterEach() throws Exception {
        WaldotTestEnvironment.cleanup(g, null);
    }

    @Test
    public void alarmFollowsValue() {
        final Vertex v = g.addVertex("type", "example:threshold", "label", "t", "limit", "10");
        v.property("value", 20.0);
        assertEquals(true, v.property("alarm").value());
        assertEquals(1L, v.property("exceeded-count").value());
    }
}
```

See `plugins/waldot-plugin-generator/src/test/java` for complete examples, including OPC UA client checks with `WaldotTestClientHandler`. Property writes are processed synchronously, so the assertions can follow the write directly; only values produced by background threads need polling.

4.2.9 9. Checklist

- [ ] Plugin class annotated with `@WaldotPlugin`, top-level, public no-arg constructor, in a package under `net.rossonet.waldot`, `plugin` or `plugins`.
- [ ] Type labels with a unique prefix (`vendor:type`) to avoid clashes with `sim:*`, `rules:*` and other plugins.
- [ ] Vertex class with the standard constructor, static `generateParameters(...)`, `clone()`.
- [ ] Lowercase property keys; `super.notifyPropertyValueChanging(...)` called first.
- [ ] Threads stopped in `close()` / `notifyRemoveVertex()`; plugin state cleared in `reset()`.
- [ ] Commands created through `AbstractCreateVertexCommand` with `validate(...)` for cross-parameter checks.
- [ ] Compiled for Java 26; extra dependencies shipped in `/app/lib`.
- [ ] Behavior verified with an OPC UA client: variables, events, commands.